

סיכום הרצאות 2026 – מערכות הפעלה

מרצה: פרופסור דוד סרנה

© גיא יער-און

ייתכן שנפלו טעויות בעת כתיבת הסיכום, ולכן השימוש בסיכום על אחריותכם בלבד.

תוכן עניינים:

2	הרצאה 1: חזרה על מבנה מחשב, מבוא למערכות הפעלה
9	הרצאה 2: ניהול זיכרון, מבנה מערכת ההפעלה, system calls
16	הרצאה 3: מכונה וירטואלית, תהליכים (Process)
22	הרצאה 4: המשך תהליכים, Sockets, Threads
28	הרצאה 5: CPU Scheduling
36	הרצאה 6: Synchronization

הרצאה 1

מהי מטרת הקורס? עד היום התעסקנו ברמת תוכנית בודד ותהליך בודד: סינטקס של קוד, אלגוריתמים בהם משתמשים, כתיבת קוד נכון ויעיל וכו'. בקורס – אנחנו נתמקד בעולם בו פועל התהליך שלנו והאינטראקציה של אותו תהליך עם הסביבה, וכמובן נדון בכמה וכמה תהליכים. במהלך הקוד נדון בכמה וכמה שאלות:

- מה הן מערכות הפעלה?
- מה הן יודעות לעשות?
- כיצד מערכות הפעלה בנויות?
- מרכיבים עיקריים של מערכות הפעלה – תהליכים, Threads, CPU-Scheduling, וכו'.

נעיר מראש שבמהלך ההרצאות נדון בעקרונות כלליים של מערכות הפעלה – ולכן: בהרבה מהפעמים התשובה תהיה 'לא ניתן לדעת', שכן זה יהיה מאוד תלוי במערכת ההפעלה שתרוץ. **בתרגול:** נדון במערכת הפעלה ספציפית: Linux.

חזרה על מבנה מחשב:

- ארכיטקטורת Van-Neumann: משמשת כבסיס לרוב מערכות המחשבים המודרניות, היא מאוד מאוד פופולרית היום בעקבות יתרונותיה – אנחנו שומרים את הקוד ואת הdata שלנו באותו מקום בזיכרון, מה שהופך את התכנון והיישום של תוכנה וחומרה לפשוטים יותר. כמו כן, הארכיטקטורה נתמכת ע"י רוב שפות התכנות והכלים המודרניים. עם הזמן – התגלו בשנים האחרונות מגבלות בארכיטקטורה זו.
מגבלותיה במערכות מודרניות:
- 1. מחשוב מקבילי – מתאימה יותר למחשוב סדרתי, בעוד מערכות מודרניות דורשות יותר יכולת של תכנות מקבילי עם כמה וכמה מעבדים בו זמנית.
- 2. אבטחת מידע – העובדה שהקוד והנתונים נשמרים באותו מקום בזיכרון הוא פתח עבור "פולשים" שיוכלו לגנוב לי קוד.

במערכת ישנם 4 מודולים מרכזיים:

1. Memory – זיכרון: מאחסן את כל ההוראות ואת כל הdata של התוכנית שלנו. יש לנו בו את Control-Unit שאחראי על הרצת התוכנית שלנו ואת הAlu: אחראי על פעולות חשבון ולוגיקה לפי דרישות התוכנית וכן יש לנו בו את הIO לתקשורת עם "העולם החיצון".
- הזיכרון הוא אוסף של הרבה מאוד תאי זיכרון, כאשר גודל כל תא הוא קבוע. יחידת הזיכרון הבסיסית היא bit ותא זיכרון לרוב הוא byte בודד. הוא יקר ביחס לדיסקים וכדומה.
- הזיכרון עליו אנחנו מדברים נקרא RAM (Random Access Memory): מדובר בזמן גישה זהה לכל תא זיכרון. מדוע זה טוב לנו? כי זה גורר שלא אכפת לנו היכן התוכנית תשב בזיכרון – זמן הגישה זהה.
- WORD:** אוסף כל כמה ביטים (בהתאם לארכיטקטורה של המעבד), מדובר ביחידת המידע שמועבדת בפעולה בודדת. זהו גודל שלרוב זהה לגודל הרגיסטרים ולBUS. לרוב מדובר ב8 ביטים.
- לכל תא זיכרון ישנה כתובת:** את הכתובת אנחנו צריכים על מנת שנדע היכן הנתון נמצא בזיכרון. את הכתובות מייצגים באמצעות מחרוזת של ביטים. בהינתן n ביטים, נוכל לתמוך בזיכרון בגודל של $2^n - 1$ כתובות. מדובר בכמות המקסימלית של זיכרון שנוכל לשים במערכת שלי, כיוון שלא נוכל לייצג עוד זיכרון בשיטה זו.

סיכום מערכות הפעלה | © גיא יער-און

- את גודל הזיכרון מייצגים בבייטים – KB,MB,GB,TB וכו'.
- Hz הוא מס' הגישות לזיכרון שאנחנו יכולים לעשות בשנייה. כלומר 1 Hz הוא מחזור (Cycle) אחד בשנייה. מהו סייקל? יחידת העבודה הקטנה ביותר של זיכרון. הזיכרון יכול לבצע פעולה בכל רגע נתון. ככל שמס' הסייקלים גדול יותר פר שנייה הזיכרון עובד מהר יותר כי הוא מספיק לבצע יותר פעולות.
- מהירות הוא הנתון שאומר כמה סייקלים הזיכרון מבצע בשנייה אחת.

במקום לענות על השאלה: "כמה סייקלים יש בשנייה", נוכל לחשב כמה שניות לוקח סייקל אחד בודד:

$$\text{Time Per Cycle} = 1/\text{Hz}$$

לצורך ההמחשה: אם משהו קורה פעמיים בשנייה, כלומר 2 Hz, אז די ברור שכל פעם כזו (סייקל) לוקח חצי שנייה. כלומר, $\frac{1}{2}$.

פעולות על הזיכרון:

מערכת הזיכרון מופעלת באמצעות:

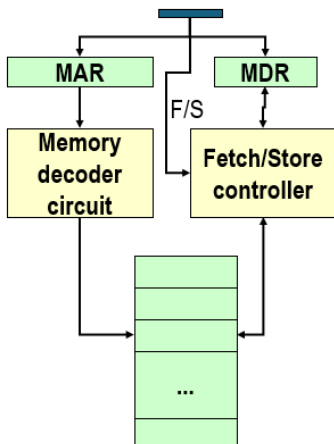
1. רגיסטר כתובות זיכרון – MAR – בעת שליפה ישמש לקרוא את הערך שאנחנו קוראים מהזיכרון.
2. רגיסטר נתוני זיכרון – MDR – ישמש לקרוא את הערך שנרצה לכתוב בזיכרון.
3. אות לשליפה/אחסון – אות חשמלי על מנת להבין האם הפעולה הבאה תהיה שליפה או אחסון.

- **שליפה – Fetch:** שליפת עותק של תוכן בתא הזיכרון עבור הכתובת שצוינה. מדובר בפעולה לא הרסנית כי היא משמרת את הערך בתא הזיכרון.

1. טוענת את הכתובת לתוך MAR

2. מבצע Decode לכתובת הזו: יחידת הזיכרון מפענחת את הכתובת שבMAR כדי לאתר פיזית את התא המתאים בזיכרון, היא תעתיק את התוכן של תא הזיכרון לתוך MDR
- אחסון – Store:** אחסון הערך שצוין לתוך תא הזיכרון עבור הכתובת שצוינה. פעולה הרסנית, דורסת את הערך הקודם בתא הזיכרון.

1. טוען את הכתובת לתוך MAR: המעבד מציב שם את כתובת התא אליו הוא רוצה לכתוב.
2. טוען את הערך לתוך MDR: המעבד מציב שם את הערך החדש שהוא רוצה לאחסן.
3. מבצע Decode של הכתובת לMAR: יחידת הזיכרון מפענחת את הכתובת בMAR על מנת לדעת להיכן לכתוב.
4. מעתיק את התוכן של MDR לתוך תא זיכרון עם הכתובת הספציפית שנבחר וכך מחליף את המידע הישן.



מודול Input/Output (IO):

מטפל בהתקנים שמאפשרים למערכת המחשב לתקשר וליצור אינטראקציה עם העולם החיצון – לתקשר וליצור אינטראקציה עם העולם החיצון (מסך, מקלדת), לאחסן מידע.

לכל התקן של IO ישנו התקן בשם **IO CONTROLLER**: הבעיה היא שמהירות התקני IO נמוכה מאוד ביחס למהירות הRAM – ולכן בכל שימוש בIO נעכב את כל המערכת, לכן הפתרון הוא שימוש בהתקן זה. מדובר במעבד ייעודי שכולל **רכיב זיכרון קטן (IO Buffer)**: יש בו לוגיקת בקרה לניהול התקן הקלט/פלט: הוא ינהל עבורנו בצורה עצמאית את האינטראקציה עם המכשיר עצמו. למשל: הזזת זרוע הדיסק. היתרון: כיוון שהוא עצמאי, אפשר לבצע את הפעולות שלו במקביל למעבד שלי שמבצע פעולות של התוכנית. בסיום ניהול האינטראקציה

סיכום מערכות הפעלה | © גיא יער-און

עם device, ה-CONTROLLER יודיע לי על כך. ישנם כמה וכמה Controller's: אחד ייחודי לכל סוג של מכשיר. למשל – ישנו Device Controller עבור המסך שיכול לנהל כמה וכמה מסכים.

מודל ה-ALU:

מיועד לבצע פעולות מתמטיות, לוגיות. במחשבים של היום ה-ALU משולב כבר בתוך ה-CPU. הוא מורכב ממעגלים לביצוע הפעולות האריתמטיות/לוגיות, רגיסטרים לאחסון תוצאות חישוב הביניים, BUS שמחבר בין השניים.

מודל ה-Control Unit (יחידת הבקרה):

- התוכנית מאוחסנת בזיכרון כפקודות בשפת מכונה (לאחר שהקומפיילר הפך אותם לפקודות בשפת מכונה).
- תפקידה של יחידת הבקרה הוא לבצע תוכניות על ידי חזרה על השלבים הבאים:
 1. שליפה – Fetch: שליפת הפקודה הבאה לביצוע מהזיכרון
 2. פענוח – Decode: קביעת הפעולה שיש לבצע
 3. ביצוע – Execute: ביצוע הפעולה ע"י שליחת אותות מתאימים ל-ALU, לזיכרון ולתתי המערכות של הקלט/פלט. **תמיד שלב זה מתבצע מבחינת הקורס שלנו.**

הוא ממשיך לבצע את 3 הפעולות הללו, אחת אחרי השנייה: עד לפקודת עצירה.

פקודות בשפת מכונה:

פקודה בשפת מכונה מורכבת מ:

1. קוד פעולה: מציין איזו פקודה עלינו לבצע.
2. שדות כתובת – מציינים את כתובות הזיכרון של הערכים שעליהם הפקודה שלנו תפעל.

Opcode (8 bits)	Address 1 (16 bits)	Address 2 (16 bits)
00001001	0000000001100011	0000000001100100
קוד פעולה 9	כתובת זיכרון 99	כתובת זיכרון 100

לדוגמה: add x,y: אנחנו צריכים לקרוא את כתובת הזיכרון של x ואז את הכתובת של y וכן יש את ה-OPCODE של הפקודה. וזה יראה כך:

PC (Program counter) – רגיסטר ששומר את הכתובת של הפקודה הבאה בתור לביצוע. בעת שפקודה נשלפת מהזיכרון, הוא מתעדכן כדי להצביע על הפקודה הבאה בזיכרון.

IR (Instruction Register) – רגיסטר שמחזיק את הפקודה עצמה שנשלפה כרגע מהזיכרון, שומר את ה-OPCODE שעשינו לו Fetch מתוך הזיכרון.

Instruction Decoder: מפענח הפקודות. יחידה לוגית שמקבלת את הפקודה מה-IR ומפענחת אותה. הוא מתרגם את הקוד הבינארי של הפקודה לאותות חשמליים שמפעילים את הרכיבים הרלוונטיים. הוא הרכיב החומרתי שמופעל בעת ביצוע Decode לפקודה.

מבנה מערכת המחשב:

ניתן לחלק את מערכת המחשב לארבעה מרכיבים:

- אפליקציות (Applications): מגדירים כיצד יבוצע השימוש במשאבי המערכת על מנת לפתור בעיות חישוביות של משתמשים. היא יכולה להיות קומפיילר, משחק וידאו, אתר וכו'.
- משתמשים: לא בהכרח מדובר באנשים – גם מחשבים מרוחקים הם משתמשים. למשל עבור web server ישנם מחשבים שפונים אליו כבקשות ומוגדרים כמשתמשים.

סיכום מערכות הפעלה | © גיא יער-און

- מערכת ההפעלה: נמצאת מעל החומרה. היא שולטת על ומתאמת את השימוש במשאבי החומרה בין התהליכים והמשאבים השונים. מדובר על מערכות שיש בהן הרבה מאוד תהליכים שרצים – ואיננו יודעים מי מבין התהליכים יותר חשוב או יקר. עלינו רק לשלוט ולנהל אותם.
- חומרה (Hardware): ה"ברזלים". אילו משאבי מחשב בסיסיים יש לנו? CPU, Memory, IO Devices וכו'.

הגדרה: מערכת הפעלה היא תכנית אשר משמשת כחוצץ בין המשתמש של מערכת המחשב והחומרה של מערכת המחשב. מערכת ההפעלה נמצאת במכונות, במכשירים ביתיים, בטלפונים, מחשבים אישיים ותעשייתיים וכו'. מטרותיה: הרצת תוכניות המשתמש בצורה קלה, הפיכת השימוש במחשב לנוח יותר ושימוש יעיל יותר בחומרת המחשב.

נשים לב: מערכת ההפעלה מוגדרת כתוכנית גם כן – ולכן דורשת משאבים גם בעצמה. כלומר: גם כשלכאורה המחשב נראה שלא עושה כלום, המעבד מריץ קוד של מערכת ההפעלה.

שאלה. האם יכולה להיות מערכת מחשב ללא מערכת הפעלה? (כלומר, אם אפשר לקנות מחשב ולא להתקין עליו מערכת הפעלה).

תשובה. בתכלס, כן, אבל. זה ייצור עבורנו הרבה מאוד בעיות:

- נצטרך לכתוב תוכניות בקוד מכונה.
- לא נוכל להריץ יותר מתוכנית אחת בו זמנית.
- נצטרך לטפל בעצמנו בכל המקרים והתגובות – שגיאות, תזוזת עכבר, לחיצה על מקשים וכו'.
- נצטרך לנהל אינטראקציה עם החומרה בעצמנו.

מתי כן נראה את זה? במערכת סגורה – שבה אין אינטראקציה עם האדם, שהופך את התנהגות התוכנית ללא דטרמיניסטית. הרי, אם המערכת סגורה וידועה והכל בה דטרמיניסטי אז אין לנו צורך במערכת הפעלה (שדורשת הרבה משאבים).

סוגי קטגוריות מחשוב:

- **Supercomputer:** מחשב על. מחשב נמצא בחזית יכולת החישוב, במיוחד במהירות החישוב. מחשב מאוד יקר אותו מדינות/אוניברסיטאות קונות.
- **Mainframe:** מחשבים חזקים (מבחינת אמינות, זמינות וביצועים). משמשים בעיקר ארגונים גדולים ליישומים חשובים – כמו רישום אוכלוסייה, ניהול מלאי וכדומה. מהירות פחות גבוהה משל מחשב העל, אך עדיין גבוהה.
- **Personal Computer (PC):** כל מחשב רב תכליתי שגודלו, יכולותיו ומחירו המקורי הופכים אותו לשימושי עבור משתמשים פרטיים. אתה מוכר אותו עבור אדם ואינך יודע מה הוא יעשה בו – אם הוא ישתמש בו עבור מוזיקה בנטפליקס, או שיכתוב בו קוד.
- **Handheld:** דומה ל-PC, מכשיר מחשב אך עם סוללה שניתן לתפעול ביד אחת. הוא כולל בדרך כלל מסך תצוגה עם קלט של מגע. (כמו טלפון סלולרי).

לכל מערכת הפעלה יש מטרות ועיצוב שונים. למשל:

- **Mainframe:** הפוקוס של מערכת ההפעלה יהיה על ניצול מקסימלי של משאבי חומרה – פחות נוחות המשתמש, יותר ניצול מקסימלי של החומרה.
- **PC:** תמיכה מקסימלית בהרצת תוכניות של המשתמש – נוח עבור המשתמש.

UI-User Interface: המעטפת הוויזואלית והאינטראקטיבית שדרכה המשתמש מתקשר עם המכונה (כמו כפתורים, תפריטים ועיצוב המסך).

סיכום מערכות הפעלה | © גיא יער-און

- **Handheld**: שיהיה נוח להריץ אפליקציות, כי אין מקלדת/עכבר. נרצה גם ביצועים טובים פר יחידת ניצול של הסוללה.

בבירור – הנוחות מגיעה על חשבון היעילות. כמו תמיד, נאלץ לוותר על חלק מהיעילות עבור הנוחות ולוותר קצת על הנוחות עבור היעילות.

תפקידיה של מערכת ההפעלה:

- **Resource allocator**: בדומה לממשלה, שמקצה כמה מהתקציב ילך לכל תחום (חינוך/בריאות וכו'), מערכת ההפעלה מנהלת את המשאבים, היא מחליטה במצבים של קונפליקט על מנת שהשימוש במשאבים יהיה יעיל והוגן.
- **Control program**: שולטת על ההרצה של התוכניות השונות על מנת למנוע שגיאות, ולמנוע שימוש מוטעה במערכות המחשב.

תפקידיה חשובים מאוד בעיקר כאשר ישנם כמה משתמשים שמחברים לאותו mainframe.

שאלה. האם צייר, סוליטייר וכו' נחשבים כחלק ממערכת ההפעלה?

תשובה. אין תשובה אחת. יש כאלו שסוברים שכן – כל מה שהגיע בהתחלה כחלק מדיסק ההתקנה של מערכת ההפעלה הוא חלק ממערכת ההפעלה, ויש כאלו שסוברים שלא, והם לא חלק ממערכת ההפעלה.

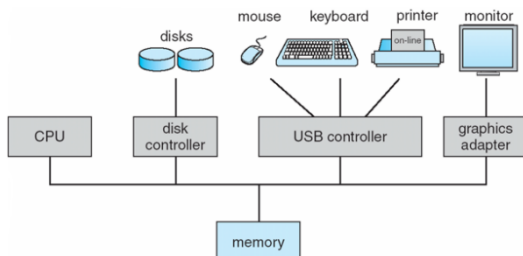
הגדרה: התוכנית/תהליך שרץ כל הזמן במערכת המחשב נקרא kernel. כל שאר התהליכים שרצים ע"פ דרישה מקוטלגים כ system programs (תוכניות שהגיעו יחד עם מערכת ההפעלה). למשל – הצייר מקוטלג כך. הוא הגיע עם מערכת ההפעלה, אך לא רץ כל הזמן. לעומת זאת application programs אלו דברים כמו chrome, הורדנו למחשב אך זה לא הגיע עם מערכת ההפעלה. כאשר אפליקציה רוצה לקרוא קובץ, להפעיל מצלמה, להקצות זיכרון – kernel מטפל בזה. הקרנל נותן יכולות בסיסיות, ותוכניות המערכת הופכות אותן למשהו נוח לשימוש.

Middleware frameworks: מדובר בשכבת ביניים שמטרתה היא לתת למפתחי אפליקציות שירותים מוכנים מבלי שיצטרכו להתעסק ישירות בפרטים הנמוכים של הקרנל. זה עדיין יעבור פונקציונאליות של הקרנל.

הפעלת המחשב: נניח וקנינו מחשב חדש, עליו כבר מותקנת מערכת ההפעלה. לחצנו על כפתור ההדלקה. מה קורה? ברגע שהדלקנו את המחשב ישנו תהליך של reboot. עולה **reboot program**: מדובר בתוכנית דטרמיניסטית שסגורה על עצמה ויודעת להתקדם קדימה ללא עזרה מהמשתמש – היא מאוחסנת ב-EEPROM (רכיב חשמלי, לא נמחק בניגוד ל RAM, בעת שנסגור את המחשב ונדליק יום למחרת הוא יישאר שם). תוכנית זו מאתחלת את כל מרכיבי המערכת – תוכן הזיכרון, הרגיסטרים של המעבד וכו'. בנוסף: היא זו שמעלה את kernel של מערכת ההפעלה ומעבירה אליה את השליטה. בנקודה זו – מערכת ההפעלה הופכת לעצמאית, כל הקוד שלה טעון ומרכיביה מאותחלים. ואז – מערכת ההפעלה לא עושה כלום: היא מחכה, עד שיהיה אירוע כלשהו: שהמשתמש יפעיל את העכבר, יזיז משהו וכו'.

מעבדים:

מערכת ההפעלה מאופיינת ע"י מעבד (אחד או יותר, כיום יש כמה וכמה) Device 1 controllers: כמו Disk controllers controller וכו'. כל אחד מהם אחראי על סוג מסוים של devices. הם מחוברים באמצעות common bus שמאפשר להם גישה לזיכרון משותף, כך:



כל device controller אחראי על סוג מסוים של device. המחשב לא יודע כיצד לדבר ישירות עם המנוע של הארד דיסק, ולכן הוא מדבר עם הבקר והבקר יודע כיצד לנהל את המכשיר הספציפי שמוטאם לו.

סיכום מערכות הפעלה | © גיא יער-און

המידע מ ואל device מנוהל באמצעות Local Buffer: המעבד מעביר מידע מ/אל הזיכרון הראשי אל הלוקאל באפר. כיוון שהתקני IO איטיים מאוד בהשוואה למעבד, המידע לא יעבור ישירות מהדיסק למעבד. הלוקאל באפר הוא זיכרון מקומי קטן. המידע מהמכשיר נאסף קודם כל בבאפר של הבקר, כשהמידע מוכן הוא מועבר אל הזיכרון הראשי, ומשם למעבד (ולהפך). במקום שהמעבד ישב וישאל כל הזמן "סיימת? סיימת?" – הוא בונה על כך שבעת שהdevice controller יסיים הוא ישלח interrupt: לכן הוא יכול להמשיך לבצע פעולות אחרות תוך כדי.

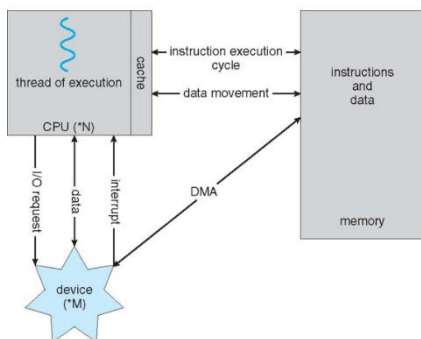
ברוב המערכות נמצא לפחות מעבד אחד מסוג **general purpose**: מעבד שיועד לעשות הכל, הוא לא נבנה בצורה שנותנת יתרון ליישום מסוים – הוא נבנה בצורה כללית ויועד לבצע את הכל. לצד מעבד זה, מוסיפים מעבדים נוספים מסוג **special purpose**: למען מטרה מסוימת. למשל – GPU (מעבד עבור גרפיקה). לצד המעבדים הנוספים, תמיד נהיה חייבים את המעבד הכללי.

מערכות **Multiprocessors** הן מערכות בהן יש יותר ממעבד אחד – ישנה תקשורת בין המעבדים, שחולקים את אותו bus ולעיתים גם את הזיכרון. מה היתרונות של מערכת שכזו?

- מהירות, כמובן.
 - כלכלי: אך למה שלא נקח מעבד פי 2 יותר מהיר, במקום לקחת 2 מעבדים? זה מגיע כמובן מבחינה כלכלית – לרוב ליצור מעבד מהיר פי 2, הרבה יותר יקר מאשר ליצור 2 מעבדים במהירות שקולה.
 - בעת שיש תקלה, אם יש לנו רק מעבד אחד תקלה במעבד משביתה את המערכת כולה. בעת שיש כמה מעבדים, הם יכולים לכפות על המעבד שהתקלקל – המהירות תרד, אך המערכת לא תושבת.
- ישנן שתי ארכיטקטורות של ריבוי מעבדים:

- **Asymmetric Multiprocessing**: משימות מסוימות מוקצות למעבדים מסוימים בלבד. בפרט, יתכן כי מעבד יחיד יהיה אחראי על טיפול בכל interrupts במערכת ואולי אף יבצע פעולות קלט/פלט.
 - **Symmetric Multiprocessing**: מתייחס לכל רכיבי העיבוד במערכת באופן זהה – אפשר לפנות לכל המעבדים בעת דרישה.
- בכל פעם שנאמר בקורס core: הכוונה למעבד. (זה לא כך בפועל, אך זה הדיון במסגרת הקורס).

- **Clustered systems**: דומה לריבוי מעבדים, אך בריבוי מעבדים ישנם כמה מעבדים שחולקים אותו לוח אם ואותו זיכרון: כאן מדובר במערכות נפרדות (בזיכרון, לוח אם) שעובדות יחד. הן חולקות אחסון משותף דרך רשת שנקראת SAN. היתרון הגדול הוא הזמינות – גם אם אחת המכונות קורסות, השירות לא יקרוס. בדומה, ישנן 2 שיטות לגישה זו:
1. **Asymmetric Clustering**: ישנה מכונה אחת שהיא במצב "Hot-StandBy": היא לא עושה כלום, והיא רק מחכה אם אחת השרתים יקרוס: היא תכנס במקומו.
 2. **Symmetric Clustering**: כל המכונות עובדות ומריצות אפליקציות במקביל, אם אחת נופלת הן מתחלקות בעומס שלה. זה יעיל יותר כמובן כי כל החומרה מנוצלת.



איך מחשב מודרני פועל? יש לו מעבד – שמדבר גם עם הזיכרון, וגם עם device. בזיכרון – יש לנו גם פקודות, וגם דטה. המעבד יהיה זה שייקח דטה מהזיכרון, ויעביר אותו אל device: באמצעות device controller.

אם יש בקשה של device מהמעבד לבצע פקודות IO הוא ישלח בקשה: IO REQUEST, לשם כך לפעמים יהיה צריך להעביר data בין CPU לdevice. בסיום – device ישלח interrupt, עליה נרחיב כעת:

Interrupt: פסיקה – מדובר באות חשמלי שמתקבל במעבד מרכיב החומרה (דיסק, עכבר וכו'). נוצר ע"י device. הוא מעביר את השליטה ל **interrupt service routine (ISR)**. זו הדרך של device לומר "סיימתי": זו הדרך שלנו לעצור את מה שעושה המעבד. המעבר שעצר, יטפל כעת בinterrupt וכשהוא יסיים נוכל לחזור לעשות מה שעשינו קודם. הוא מחייב שמירה של ה כתובת של ה interrupted instruction ושל תוכן הרגיסטרים (על מנת שנוכל לחזור בהמשך למצב שבו היינו – עם ערכי הרגיסטרים שהיו לפני ביצוע ה interrupt). להכל יהיה interrupt: בכל פעם שה device controller יסיים משהו – הוא ישלח. נבחין כי בעת לחיצה של מקש במקלדת – ה device controller יטפל בכל התהליך – יזהה איזה מקש במקלדת הוקש וכו', ורק בסוף הוא יבצע interrupt.

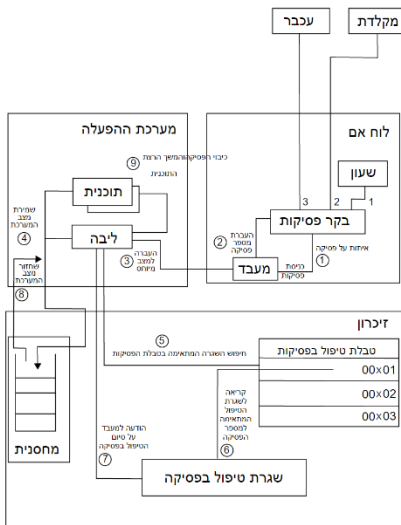
אם נסכם את התהליך, נעזר בתמונה משמאל להבנת התהליך:

שלב א': זיהוי – בלוח האם: התקן חיצוני (מקלדת/עכבר) שולח אות חשמלי לבקר הפסיקות. הבקר מעביר את האות אל המעבד. בקר הפסיקות שולח למעבד את ה"זהות" של הפסיקה (מספר הפסיקה. למשל: "פסיקה מספר 2 הגיעה דרך המקלדת")

שלב ב': החלפת הקשר – במערכת ההפעלה: המעבד מפסיק את ריצת התוכנית הרגילה ועובר ל kernel mode, על מנת שיוכל לגשת למשאבי מערכת מוגנים. כמו כן, המערכת שומרת את הנתונים הנוכחיים של התהליך (רגיסטרים, כתובות וכו') בתוך המחסנית על מנת שנוכל לחזור אליהם אחר כך ללא איבוד מידע.

שלב ג': איתור וביצוע – בזיכרון: חיפוש בטבלת הפסיקות: המעבד נגש לטבלת וקטורי הפסיקות בזיכרון ומחפש את הכתובת של הקוד שמתאים למס' הפסיקה שהתקבל בשלב 2. המעבד קופץ לכתובת שבטבלה ומצא, ומריץ את שגרת הטיפול בפסיקה (ISR), זהו הקוד שבאמת מטפל בלחיצת המקלדת או בתזוזת העכבר.

שלב ד': חזרה לשגרה – הודעה על סיום, המערכת מסיימת ומעדכנת את המעבד שהטיפול הושלם. שחזור מצב המערכת – שולפת מהמחסנית את הנתונים ששמרה ומחזירה אותם לרגיסטרים. המעבר חוזר למצב User Mode וממשיך להריץ את התוכנית בדיוק מהנקודה שבה נעצרה.



הרצאה 2

Trap: דומה לinterrupt, הוא לא אות חשמלי אלא הודעה שמעביר קוד שלנו. הוא software-generated interrupt. פונקציונלית, הוא גורם למערכת ההפעלה להתעורר ולעשות משהו, לכן הוא דומה לinterrupt, הוא נגרם בעקבות שגיאה/בקשה של תוכנית של המשתמש.

Interrupt Handling: כאשר נקבל interrupt ישנן 2 גישות עיקריות כיצד לטפל בו, ראשית עלינו להבין מה סוג הinterrupt שקרה. הגישות:

- **Polling:** יצרנו מראש קוד גנרי שיודע לטפל בכל הinterrupt שיגיעו (יודע לתת את כל התגובות לכל המקרים). ואז – כשנקבל interrupt נריץ את הקוד הגנרי ונבין באמצעות הקוד הגנרי מה קרה.

סיכום מערכות הפעלה | © גיא יער-און

- **Vectored interrupt system:** יחד עם interrupt נקבל מס' כלשהו (למשל: מס' 7 הוא הודעה של המדפסת, מס' 3 הוא הודעה מהשעון וכו'). באמצעות המס' הנ"ל אנחנו יכולים ללכת אל טבלה בזיכרון – להגיע אל המקום המתאים בטבלה, ושם נמצא את כתובת הקוד שצריך לטפל בinterrupt הנ"ל.

בבירור, לכל interrupt יהיה קטע קוד רלוונטי (routine), ששמור בזיכרון. מהם היתרונות/חסרונות של כל אחת מגישות הטיפול?

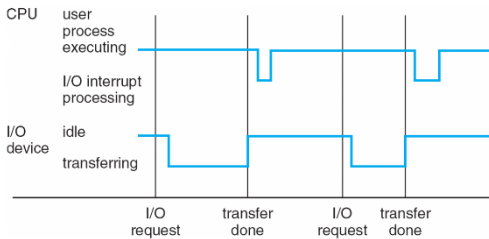
בגישת Vector: נקבל שזמן הטיפול יהיה הרבה יותר מהיר – שכן נמצא את קוד הטיפול הרבה יותר מהיר (זו מעין "טבלת האש"), אך – כנראה שיש הרבה כפילויות: טיפול בinterrupt שהגיע מדיסק מסוג A interrupt שהגיע מדיסק מסוג B – יתכן חפיפה בקוד שלכאורה יכולנו לאחד אותם אם היה בידינו קוד מסוג Polling, ולכן אנחנו משתמשים בהרבה יותר זיכרון כי יש קודים נפרדים לכל אחד מהמקרים.

כיוון שהמהירות היא קריטית (המעבד עוצר, מחכה..) – רוב המערכות עובדות בגישת Vector.

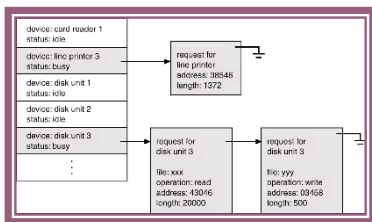
טיפול ב-I/O:

סינכרוני: ברגע שהתוכנית של המשתמש מתחילה לרוץ, אנחנו מעבירים את השליטה לKernel שיפעיל את device controller ובאמצעותו את device עצמו, ועד שלא סיימנו את הטיפול ב-I/O התוכנית של המשתמש לא תמשיך לרוץ

אסינכרוני: מבקשים את I/O ממערכת ההפעלה באמצעות device controller ומיד מחזירים את השליטה לתוכנית של המשתמש – ברגע שהdevice יסתיים הוא ישלח interrupt. אך, נבחין כי יתכן ויהיה תלות להמשך קוד המשתמש ב-I/O: ולכן במקרים אלו נעבוד בצורה סינכרונית. אם רק רוצים להדפיס למסך למשל – ואין תלות בהמשך הקוד ב-I/O: נעבוד בגישה אסינכרונית.



משמאל – דוגמה למצב אסינכרוני: המשתמש רץ עד לקו השחור הראשון, שם המשתמש מבקש I/O (למשל: לפתוח קובץ). במצב זה – מדובר בsystem call והשליטה עוברת למערכת ההפעלה – היא מריצה קוד בו היא מעבירה לdevice controller את המידע לריגיסטרים שהוא זקוק (זה החלק בין שני השחורים ש"נופל"), במקביל – כיוון שאנחנו באסינכרוני, תוכנית המשתמש ממשיכה לרוץ, עד לקו השחור ששם device controller סיים ושולח interrupt למעבד – ועובר להריץ קוד שמטפל באותו interrupt (זה המעבד) – זה ה"נפילה" בין הפס השחור השני לשלישי. לאחר מכן – התוכנית ממשיכה לרוץ (וקורים תהליכים דומים).



- מה קורה במצב אסינכרוני שבו לא סיימנו לטפל בבקשה שהגיעה, וכבר הגיעה עוד בקשה לdevice? לשם כך משתמשים במעין "תור", ישנה טבלה: Device-Status Table – כמו באיור משמאל. לכל device תהיה מעין שורה בטבלה, ובה הפניה ל"תור" בקשות.

ניהול הזיכרון:

מדוע הזיכרון כל כך חשוב? instructions חייבות להיות בזיכרון על מנת שנוכל לבצע אותן, וגם data חייב להיות בזיכרון. ניהול זיכרון עוסק בהחלטה על מה יהיה/ישהה בזיכרון בכל רגע ורגע. מדוע שלא נשמור הכל בזיכרון בכל רגע נתון? כי הזיכרון ממש יקר – ומוגבל במקומו, ולא נוכל להחזיק הכל בזיכרון בכל רגע נתון.

תפקיד מערכת ההפעלה בנוגע לניהול הזיכרון:

- היא רושמת ועוקבת על החלקים בזיכרון שכרגע נמצאים בשימוש.
- מחליטה על העברה של תוכן זיכרון שמשויך לתהליך מסוים – מ/אל הזיכרון: זה נקרא swapping. למשל: להעביר אל דיסק. מדוע שנעשה זאת? כי לפעמים אין מספיק מקום בזיכרון.

סיכום מערכות הפעלה | © גיא יער-און

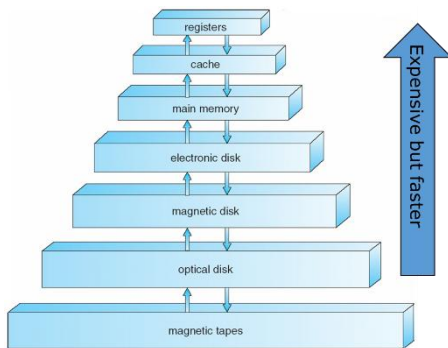
- מקצה זיכרון לתהליכים, ולוקחת זיכרון מתהליכים, ויכולה לשפר זיכרון לתהליכים (לתת עוד זיכרון).

Mass Storage Management: פרט לזיכרון הראשי (אמצעי אחסון הגדול היחיד שהמעבד יכול לגשת אליו בצורה ישירה), ישנם דיסקים. מדוע שנשתמש בדיסקים?

- נרצה לאחסן נתונים שלא יכולים להכנס ל-RAM בגלל גודלם.
- נרצה לאחסן נתונים שיהיו שם גם לתקופה ארוכה – גם אחרי שנכבה את המחשב.

ניהול נכון של Mass storage (הדיסקים) משפיע מאוד על המהירות הכוללת של פעילות המחשב – כיוון שגישה לדיסק יקרה מאוד.

תפקיד מערכת ההפעלה בהקשר: מנהלת את המקום הפנוי, מקצה מקום אחסון ומתזמנת פעולות דיסק.



משמאל: היררכיית Storage. ככל שנעלה בהיררכיה, התקן האחסון שנפגוש יהיה יותר מהיר, העלות שלו תהיה הרבה יותר יקרה פר כמות אחסון. משלב מסוים גם הזיכרון הופך לנדיף – הרגיסטרים והקאש נדיפים (נדיף הוא זיכרון שזקוק לזרם חשמלי רציף כדי לשמור על המידע שנמצא בתוכו. ברגע שמכבים את המחשב או שיש הפסקת חשמל – כל המידע שלא נשמר על הדיסק הקשיח נמחק לצמיתות).

שאלה. האם יתכן ואדם יגיע יום אחד – ויאמר שהצליח ליצור רמה בהיררכיה שנמצאת גבוהה יותר? שהיא גם מהירה יותר וגם זולה יותר?

תשובה. כן: יכול להיות, אבל מה ההשלכות? היא תעלים מההיררכיה את כל מי שנמצא מתחתיה – כי לא נצטרך להשתמש בהם יותר. הרי היום משתמשים בדיסקים כי הם זולים יותר, למרות המהירות הלא טובה שלהם. אם נמצא זאת: לא נצטרך יותר להשתמש בכל מי שמתחתיו בהיררכיה. הערה: הגודל הפיזי של הרכיב – גם עשוי להיות שיקול (שכן אם יגיע אדם ויציע טכנולוגיה כזו – רק שהיא דורשת חדר שלם לאחסון, זה יהפוך לשיקול).

Caching: העתקת מידע למערכת אחסון מהירה יותר. למשל: אם נרצה לקחת מקובץ שנמצא בדיסק פעולת חישוב 2+3, זה יעבור מהדיסק לזיכרון הראשי, אל הקאש ואל הרגיסטרים.

Direct Memory Access Structure: DMA

נזכר בתמונה מעמוד 7 – הופיע שם DMA: כעת נסביר עליו.

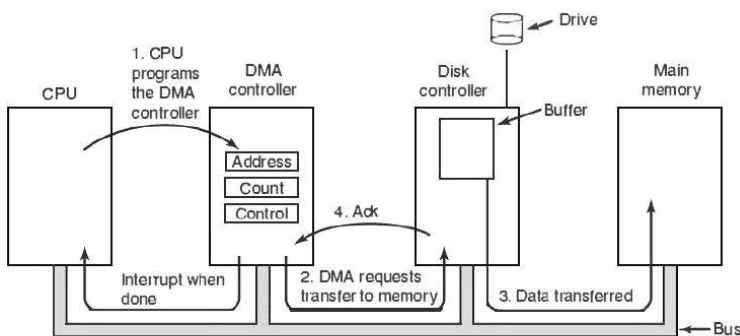


Figure 1. Operation of a DMA transfer.

המטרה של DMA היא "לעקוף" את המעבד. בשיטה הרגילה – המעבד צריך לקרוא כל בייט מהdevice ולכתוב אותו לזיכרון: מדובר בפעולה בזבזנית. כעת, דמא מאפשר להתקן קלט/פלט להעביר בלוקים של מידע ישירות לתוך ה-RAM מבלי שהמעבד יצטרך לגעת בנתונים בדרך.

בתמונה משמאל נראה תיאור של התהליך בו המידע עובר ישירות מהdevice אל הזיכרון הראשי.

שלב ראשון: המעבד לא רוצה להתעסק בהעברה של כל בייט ובייט – לכן הוא ניגש אל ה-DMA controller ומטעין אותו בנתונים – כתובת (לאיזה כתובת בזיכרון לשפוך את המידע), כמה מידע להעביר, control: סוג הפעולה (קריאה/כתיבה).

שלב שני: ה-DMA controller פונה ל-Disk controller ומבקש ממנו להתחיל להעביר נתונים.

סיכום מערכות הפעלה | © גיא יער-און

שלב שלישי: בקר הדיסק שולף מידע מה-Drive ושם אותו בבאפר: משם המידע עובר על ה-bus ישירות לזיכרון הראשי. המעבד לא רואה את המידע הזה עובר ולא נוגע בו – הנתונים נשלחים ישירות לזיכרון.

שלב רביעי: Ack – האישור: בכל פעם שבלוק מידע עובר בהצלחה מהבקר לזיכרון נשלח סיגנל Ack חזרה ל-Dma controller בשביל לעדכן את count (כמה נותר להעביר). כאשר count=0 ה-Dma controller שולח Interrupt למעבד – רק עכשיו המעבד עוצר לרגע, מבין שהמידע מוכן בזיכרון הראשי ויכול לעבוד איתו.

Multiprogramming: היום, במחשב שלנו בבית אנחנו מריצים כמה תהליכים בו זמנית (למשל: פותחים סביבת פיתוח ומתכנתים, ותוך כדי וורד פתוח, ומאזינים לשיר בנגן וכו') – הרבה פעמים נוצר מצב בו כמות התהליכים גדולה מכמות המעבדים. זהו מצב של multiprogramming. כאשר יש תוכנית בודדת שרצה, אי אפשר לשמור על המעבד והתקנים השונים בנצילות מלאה. אנחנו מעוניינים לשפר את הנצילות של המעבד – ולכן אידיאלית נרצה שבכל רגע נתון תהיה תוכנית שרצה על המעבד. multiprogramming מאפשר כמות תהליכים גדולה מכמות המעבדים, בצורה של שיתוף: כשתהליך מסוים למשל פונה לIO או מסיים הרצה, מתפנה מקום לתוכנית אחרת. ולכן ישנו מעין תור של תהליכים – צריך לנהל זאת כמובן: אך בכל פעם תהליך אחר מקבל זמן מעבד.

Timesharing: עדיין מדובר בהרצה של כמות תהליכים שגדולה מכמות המעבדים, אך דואגים להחליף אותם על גבי המעבדים בתדירות כל כך גבוהה כך שכל התהליכים שהמשתמש מריץ ברגע זה רצים כרגיל – והוא לא רואה אף אחד מהם נתקע או מחכה שיתפנה זמן מעבד. תחושת המשתמש היא שכל התהליכים רצים ברגע נתון, גם אם הם לא באמת מקבלים כל שניה זמן מעבד. נדגיש כי:

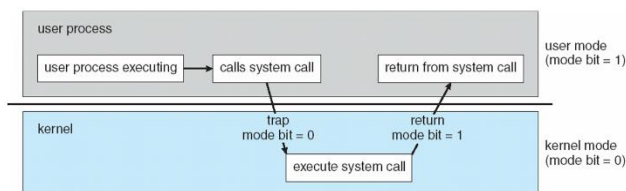
- זמן התגובה חייב להיות פחות משנייה אחת.
- לכל משתמש יש לפחות תוכנית אחת בהרצה בזיכרון.
- אם לא ניתן להכיל את כל התהליכים בזיכרון – מתחיל swapping בין התהליכים.

מעבר בין User mode לKernel mode:

מערכת ההפעלה צריכה לפעמים להריץ קוד שלה על מנת לבצע משהו עבור תוכנית של המשתמש – למשל ב-system call. הקוד של מערכת ההפעלה הוא יותר בטיחותי כי ישבו עליו הרבה ודיבגו אותו והוא לא ינסה לעשות דבר זדוני. היינו רוצים שהקוד הזה יעשה יותר. ישנן פקודות שנרצה להגדירן כprivileged: פקודות שרק מערכת ההפעלה יכולה לבצע – המשתמש לא.

כיצד נדע האם אנחנו ב-User Mode? ישנו ביט מיוחד mode bit: אם ערכו 1 אנחנו ב-user mode, ואחרת אם הוא 0 אנחנו ב-kernel mode. אם אנחנו ב-user mode – לא נאפשר להריץ פקודות שהוגדרו כprivileged.

בעת שנקרא ל-system call, נשלח trap למערכת ההפעלה וה-mode bit הופך להיות 0. מכאן: השליטה עוברת למערכת ההפעלה. בסיום, ה-mode bit הופך חזרה להיות 1: וחוזרים ל-user mode.



מבנה מערכת ההפעלה

שאלה. מה הופך מערכת הפעלה להיות "מערכת הפעלה"? הרי – לכל מכשיר יש מעין ממשק למשתמש.

תשובה. איפה שיש ממשק משתמש בלבד – כמו בכפתורים במיקרוגל או במכונת הכביסה, במקומות כאלו הממשק הוא מעין שכבת תקשורת בין משתמש למכשיר. הממשק הנ"ל לא מייצר ניהול משאבים – כפי שנותנת מערכת ההפעלה. למעשה, מכשירים פשוטים

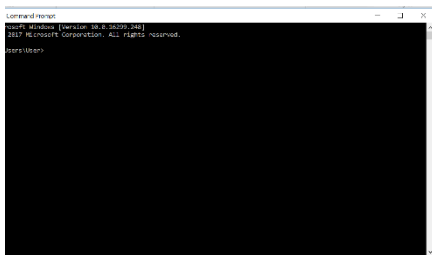
סיכום מערכות הפעלה | © גיא יער-און

יכולים לפעול עם תוכנה דטרמיניסטית פשוטה שמנהלת מס' פעולות מוגבלות מוגדרות בראש ואין צורך שיתעסקו בניהול משאבים. כלומר: למערכת הפעלה יש עושר בניהול משאבים ובמורכבות של ניהול המשאבים.

Director: סט של הוראות מחשב – שמגדיר פעולה רציפה שמורכבת מכמה וכמה פעולות. מדובר בראשית של מערכות ההפעלה. לפני שהופיע director הכל היה מורכב מהתערבות ידנית של מפעילים אנושיים – היה צריך להזין/לפקח/לקבוע דברים שונים. לאחר כניסת director (שנות 60) – היה שינוי גדול, היה אפשר להפעיל דברים הרבה יותר מורכבים שקודם בני אדם לא יכלו לפקח עליהם (או שהיה קשה לפקח עליהם).

Services שמציעה מערכת ההפעלה

ממשק משתמש – User interface (UI): כמעט לכל מערכת הפעלה יש UI (למערכות ישנות אין). לרוב נראה כמה וכמה סוגים בכל מערכת הפעלה. וישנם כמה סוגים שונים:



- **CLI: Command Line interface** – המשתמש מזין פקודה, ה-CLI לוקח את הפקודה (fetch) ומבצע אותה – הופך אותה לרצף של system calls.

כיצד מממשים CLI?

1. כחלק מ-kernel: תמיד נמצא בזיכרון, וה-`command interrupter` קופץ למקום המתאים בזיכרון.
 2. לכתוב תוכניות, כך שכל תוכנית מממשת פקודה אחרת (ושם התוכנית יהיה זהה לשם הפקודה) ולשמור את התוכניות בדיסק. למשל – אם ב-Unix נבצע `rm file.txt` – המערכת תחפש קובץ בדיסק שקוראים לו `rm`: בתוך הקובץ היא תמצא תוכנית – תעשה loading לתוכנית לזיכרון, ותשתמש ב-`file.txt` כקלט לתוכנית שרצה.
- מהו השיקול לבחירת המימוש? אם נשמור בדיסק – יהיה חפיפה בין קטעי קוד. הדילמה זהה לשקילות על וקטור לעומת Pooling. מהירות לעומת תחזוקה.

- לעיתים נמצא מס' interpreters במערכת הפעלה: ואז נקרא להם shell.
- **GUI:** ממשק משתמש גרפי, הוא נותן את המטאפורה של "שולחן עבודה", עכבר, קיצורי דרך (גרירה עם העכבר, משיכה, עבודה עם המקלדת וכו').

ברוב המערכות נמצא גם CLI וגם GUI – מדוע בעצם? ה-CLI הרבה יותר מהיר – ובאמצעות GUI צריך לבצע הרבה יותר פעולות על מנת לבצע את אותה פקודה פשוטה דרך ה-CLI. אך: ה-GUI הרבה יותר פשוט ואין סינטקס.

- ישנם עוד ממשקי משתמש רבים – למשל VUI: Voice User interface, ממשק משתמש מבוסס מגע, ממשק משתמש מבוסס מציאות מדומה – למשל VR Interface וכו'.

היכן עוד מסייעת מערכת ההפעלה למשתמש?

- הרצת תוכניות (טעינת תוכנית לזיכרון, הרצת התוכנית, סיום הרצה. כאשר התוכנית מסתיימת בצורה לא טבעית – יש להצביע על השגיאה ולטפל בה).
- טיפול ב-IO – נדרש קלט/פלט בתוכנית, נרצה שהתוכנית לא תגש ישירות כמובן – תמיד לפנות למערכת ההפעלה (הגנה על המשאב, ויעילות השימוש במשאב).
- טיפול בקבצים (נדון על כך בהמשך הקורס – קריאה מתוך קבצים, כתיבה לקבצים וכו').
- תקשורת בין תהליכים: העברת מידע בין תהליכי שפועלים באותו מחשב / שפועלים במערכות מרוחקות ומקושרים באמצעות רשת.

סיכום מערכות הפעלה | © גיא יער-און

- טיפול בשגיאות – לא רק שגיאות תוכנה, אלא גם שגיאות חומרה (בעיה פיזית בזיכרון, שגיאה בהתקני IO: למשל נגמר הנייר במדפסת / נפלה הרשת וכו'). בכל סוג כזה של שגיאה – מערכת ההפעלה צריכה לצאת לפעולה על מנת לעזור לנו בצורה היעילה יותר.
- לרוב מספקת מערכת ההפעלה Debugging facilities שמקלות על התכנית להבין מה קרה ולדבג.

כיצד מערכת ההפעלה שומרת על יעילות המערכת?

- הקצאת משאבים: כאשר ישנם כמה וכמה משתמשים מחוברים במקביל/ תהליכים שרצים במקביל – מנהלת בהתאם.
 - Accounting: ניהול רישום של השימוש במשאבים השונים לפי משתמשים לאורך ציר זמן – כל פעם שתהליך של המשתמש צריך להשתמש באיזשהו משאב, נרשם בצד באיזה משאב השתמש המשתמש שלי, מתי, וכמה זמן הוא השתמש. מדוע צריך זאת? משתי סיבות: 1. Billing – כלומר לחייב בהתאם לשימוש 2. עבור סטטיסטיקות עתידיות, שימשו את המעבד בצורה מושכלת יותר בעתיד (על מנת להכיר את המשתמש).
 - הגנה ואבטחה על משאבים:
- Protection:** בכל פעם שניגשים למשאבים של המערכת – ניגשים בצורה שהיא תקינה, יש בקרה של המערכת. למשל: אם משתמש רגיל שאיננו מנהל מערכת ינסה למחוק קובץ מערכת.
- Security:** אימות משתמש, הגנה על משאבי IO מגישה לא חוקית או זדונית. למשל: כשפותחים מחשב, נדרשים להזין סיסמה/טביעת אצבע.

System calls

מנגנון שמאפשר לתוכניות שרצות על המחשב לבקש שירותים מסוימים ממערכת ההפעלה. למשל: open,read,write,fork,exit וכו'. נרצה שמימוש של פקודות אלו יהיה דרך מערכת ההפעלה. לרוב הם כתובות ב high level language: או C או C++.

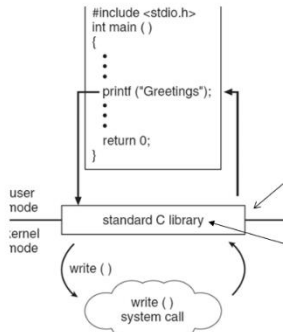
לכל system call יוצמד מספר, ותוחזק טבלה שמאנדקסת לפי מס' זה ואז בעת קריאה ל system call נגיע לשורה זו בטבלה. בעת הקריאה, יפעיל system call interface את system call המתאים ב kernel ויחזיר את סטטוס הביצוע (ואת ערכי הפלט המתאימים).

דוגמה. המשתמש לוקח קובץ, ומעתיק את התוכן שלו לקובץ יעד. כמה system calls יתבצעו?

תשובה. כרגיל, ה"כן, אבל" יהפוך ללא ניתן לדעת: בשלב ראשון, יש לשאול את המשתמש מיהם הקבצים של היעד והמקור. זה כבר דורש IO (הדפסה למסך), לאחר מכן צריך לבצע open, אם בכלל הקובץ קיים – אם לא: או שנסיים את התוכנית וזה גם system call, או שנחזור למשתמש ונבקש (IO) שוב מהמשתמש שם תקין. נניח והגענו למצב שצלחנו את זה – בידינו הקבצים הרלוונטיים: אנחנו נעבוד בלופ, נקרא קצת מהמקור ונכתוב ליעד, אך יתכן וגם בשלב זה נתקל בהרבה בעיות – אין מקום בדיסק, וכו'... נניח וגם את זה סיימנו – נצטרך לעשות close לקובץ, ולבסוף גם exit. מסקנה: יתכן שלפעולה כזו פשוטה נצטרך מאות עד אלפי system calls.

סיכום מערכות הפעלה | © גיא יער-און

לרוב לא נקרא ישירות ל system call מהתוכנית שלנו, אלא נקרא לקריאה שנמצאת ב API (Application programming interface): מגדיר את סט הפונקציות שזמינות לאפליקציה שלנו ובכלל את הפרמטרים שנדרשים כקלט לפקודה והערכים שמוחזרים להם ניתן לצפות – והוא יקרא ל system call. כלומר: אנחנו לא צריכים לדעת את המימוש עצמו של system calls אלא רק לדעת מה נדרש ממני לשלוח. נחדד, כי אם היינו קוראים ישירות ל system call – היינו צריכים לדעת את המס' המדויק של הקריאה שיכול להשתנות בין מערכות הפעלה, את אופן העברת הפרמטרים, הפורמט המדויק של הפלט וכו'. לכן נרצה להשתמש ב API. יתרה מזאת – אם יש לנו שתי מערכות מחשב שונות שמשמשות ב API: נצטרך רק להכיר סינטקטית API, זה מאפשר לנו להעביר קוד ממערכת למערכת ולקמפל אותו ולא יהיה בעיה. ללינוקס (Unix) יש את posix api.



לדוגמה, משמאל: קוד C פשוט. בפועל, בקריאת printf מדובר בקריאה ל API של ספריית C, כעת ב API: הפונקציה printf תקח את הטקסט ותעבד אותו, תכין אותו ואת הפרמטרים הנדרשים להדפסה ובסוף תקרא ל write system call. נעיר כי ה API קורה כמובן ב user mode.

```
#include <unistd.h> // עבור ספריית syscalls
#include <sys/syscall.h> // סביל את המגדרות של ספרי הקריאות (SYS_*)

int main() {
    char* msg = "Hello\n";
    syscall(SYS_write, 1, msg, 6);
    return 0;
}
```

write(1, "Hello\n", 6);

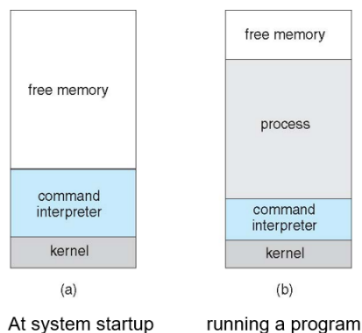
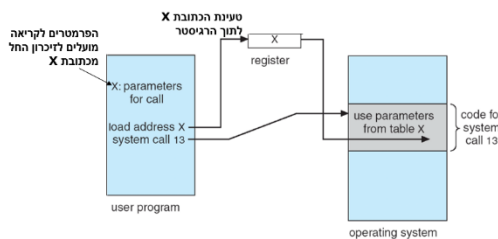
לצורך העניין, יכולנו גם לבצע את קריאת המערכת בעצמנו בצורה הבאה:

או להשתמש גם בפונקציית המעטפת write:

כאשר קוראים ל system call לרוב צריך להעביר פרמטרים ואף סט של פרמטרים. ישנן 3 צורות כלליות להעברת פרמטרים למערכת ההפעלה:

1. העברה באמצעות רגיסטרים. אך – הרבה פעמים צריך להעביר יותר פרמטרים מכמות הרגיסטרים שיש לי.
2. שמירת הפרמטרים כבלוק בזיכרון, והעברת הכתובת של תחילת הבלוק באמצעות רגיסטר.
3. שימוש במחסנית: הפרמטרים יוכנסו למחסנית באמצעות התוכנית הרצה ויימשכו ע"י מערכת ההפעלה. (pop)

לרוב, נראה את הצורה של בלוק בזיכרון. כך זה יראה בפועל (משמאל):



מערכת MS-DOS

מערכת זו פותחה לפני 60 שנים, והיא מסוג single tasking: משתמש יחיד ופרוסס יחיד.

כאן, נבחין גם לפי התמונה כי בעת ש process עלה לזיכרון, command interpreter קטן. מדוע? כי בעת שעלה תהליך מסוים – יש הרבה ב command interpreter שהופך להיות לא רלוונטי, לכן אפשר להקטין חלק ממנו לטובת זיכרון עבור התהליך. בעת שהתהליך הנ"ל יסתיים, command interpreter יטען מחדש חזרה לגודלו.

עם זאת, עברו מהר מאוד למערכת שתומכת במס' משתמשים. ישנם הרבה אתגרים שנוספו כתוצאה מכך:

סיכום מערכות הפעלה | © גיא יער-און

- אתגרי אבטחה – מניעת גישה לא מורשית למשאבי משתמשים אחרים.
- ניהול משאבים – קודם לכן היה משתמש יחיד, כעת יש כמה משתמשים ותהליכים שונים וצריך לחלק את המשאבים בצורה הוגנת.
- ניהול זיכרון מורכב הרבה יותר.
- הפרדה בין תהליכים על מנת למנוע התנגשות / הפרעה בין תהליכים.
- לשמור על יעילות הביצועים.

process D
free memory
process C
interpreter
process B
kernel

על אלו באה לענות UNIX:

כאן, כבר רצים multiprocessing, ואי אפשר להקטין את גודל command interpreter, שכן יתכן שתהליך B לא זקוק לחלק מהמידע שם, אך תהליך C כן צריך.

System programs

מדובר בתוכניות שהגיעו עם מערכת ההפעלה, ישנם המון סוגים שכאלו שמגיעים עם מערכת ההפעלה. כל מיני תוכניות שקשורות בניהול קבצים – סייר הקבצים למשל, ישנן תוכניות שנועדו לתת אינפורמציה ממערכת המחשב (למשל: שעון, לוח שנה וכו'), שינוי תוכן של קבצים (כמו notepad וכו'), קומפיילרים, דיבאגריים, browsers שמקבלים עם מערכת ההפעלה וכו'.

הרצאה 3

מימוש מערכות הפעלה

היכן נכתוב את הקוד של מערכת ההפעלה? הרי לבסוף, מערכת ההפעלה היא תוכנית.

- במקור, מערכות ההפעלה פותחו באסמבלי. בהמשך, עברו לשפות קצת יותר מתקדמות. כיום: רוב מערכות ההפעלה נכתבות ב C או C++. אך עדיין, ישנם חלקים במערכת ההפעלה שעדיין כתובים באסמבלי – החלקים שעדיין קרובים יותר לחומרה ודורשים את המהירות שמספקת אסמבלי. ה system programs (תוכניות שהגיעו עם מערכת ההפעלה) יהיו כתובות לרוב ב C++.

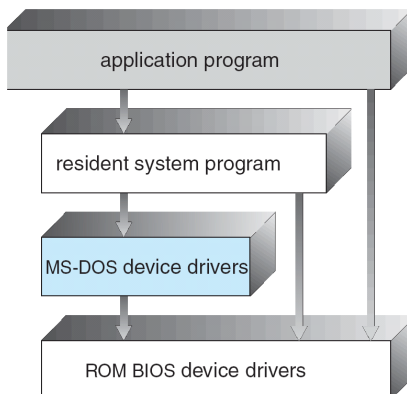
סיכום מערכות הפעלה | © גיא יער-און

כאשר אנחנו באים לתכנן מערכת הפעלה – אנחנו צריכים לייצר ארכיטקטורה מסוימת. זו שאלה שאינה פתורה: יכולים להיות הרבה עיצובים למערכת ההפעלה. אבל – ישנם עקרונות כלליים שיכולים לעזור לנו שבאים לידי ביטוי בעיצוב של מערכות הפעלה, וכן יש עקרונות שנוגעים ספציפית בנושאים מסוימים שרלוונטיים למערכות מסוימות.

- **Batch:** פונקציונליות בה אנחנו מבקשים לבצע פעולה מסוימת (למשל: פעולת חיבור) אך לא מעוניינים לראות את התהליך עצמו שחישב אלא רק את התוצאה הסופית.
- **Time shared:** ההפך, כלומר כאן נרצה לראות את התהליך עצמו שחישב בנוסף לתוצאה הסופית.

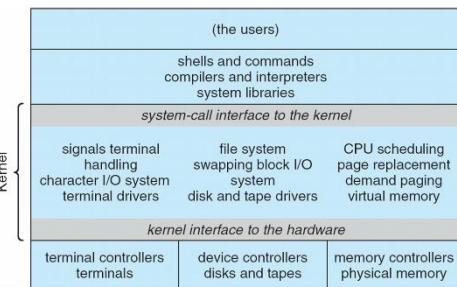
דוגמה ראשונה למבנה של מערכת הפעלה: MS-DOS:

- מערכת מאוד ישנה, נוצרה בשביל לספק פונקציונליות מקסימלית במינימום משאבים. נועדה למשתמש יחיד, לא היה אפשר להריץ כמה תוכניות במקביל, נועד לשימוש ללא חיבור לרשת וכו'.
- הארכיטקטורה שלה משמאל. apg דיברו עם תחילתה של מערכת ההפעלה – והתוכנית הנ"ל העבירה את הדברים דרך device drivers לתוכנה עצמה.
- מה הבעיה בארכיטקטורה הזו? החיצים בין הרמות שלא סמוכות זו לזו. למשל, apg ל device drivers. שכן: היה אפשר לגשת ישירות לחומרה. הבעייתיות בכך, שיכולנו למחוק למשל מהדיסק את מערכת ההפעלה עצמה. אפשר להשתמש בחומרה בצורות שלא ייעודיות לכך מלכתחילה – ולנצל בצורה זדונית. ההנחה הייתה: שרוב הדברים שנעשים נעשים בצורה תמימה ונכונה – לאט לאט הופיעו איומים והבינו שעלינו להתמודד מול תוכנה זדונית.



דוגמה שנייה: Unix

- תוכנה לתמוך בכמה משתמשים שיריצו כמה תוכניות בו זמנית. כיצד נראית ארכיטקטורה זו? משמאל. עבדה ב"שכבות". בשכבה הראשונה – קומפיילרים, interpreters, בשכבה הבאה – kernel וכו'.



Layered approach: גישה למימוש מערכת הפעלה ב"שכבות". כל שכבה תלויה בפונקציונליות של השכבה לפנייה – כאשר שכבה 0 היא כבר פנייה ישירות לחומרה ושכבה N זו הממשק למשתמש. מה יתרונותיה של גישה זו?

- **Debug** – ישנה הפרדה מלאה בין השכבות. ניתן להתחיל לדבג בשכבה מסוימת, ואם עדיין יש את הבאג הוא בהכרח בשכבות שמעליי. ניתן לצמצם את טווח החיפוש של הבעיה, ולמצוא מהר את הבעיה.

מה חסרונותיה?

- העיצוב עצמו, לרוב זה מסובך לחלק פונקציונליות של מערכת הפעלה לשכבות מסוימות. שכן, יתכן שתוך עיצוב מערכת ההפעלה נגלה שאנחנו זקוקים לפונקציונליות משכבה מעל.
- יעילות – בכל פעם צריך לגשת לכל השכבות שלפניה.

Modules

סיכום מערכות הפעלה | © גיא יער-און

רוב מערכות ההפעלה מפותחות בשיטה זו כיום, משתמשת בגישת תכנות מונחה עצמים. הרעיון הוא: במקום לבנות קרנל אחד ענק שכולל הכל מראש – או קרנל מינימליסטי שמעביר הכל למשתמשים, משתמשים במודולים:

- גישה דמוית תכנות מונחה עצמים – הקרנל מורכב מרכיבים נפרדים (כמו אובייקטים), ולכל רכיב יש תפקיד מוגדר וממשק ברור דרכו הוא מתקשר עם שאר המערכת.
- כל רכיב ליבה הוא יחידה עצמאית.
- טעינה דינמית: לא חייבים לטעון את כל הדרייברים כשהמחשב עולה, המודולים נטענים לזיכרון הליבה רק כאשר צריך אותם.

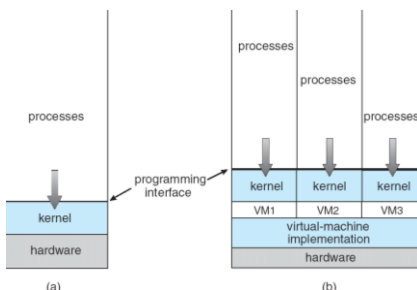
השוואה נחמדה: כשמקבלים טלפון אנחנו מקבלים את תוכניות המערכת עליו, ומתקינים בהתאם אפליקציות שנרצה. כך זה עובד בקרנל כאן – הליבה הבסיסית קטנה ומכילה רק מה שבאמת צריך, והמודולים הם קבצים נפרדים שיושבים בצד. אם למשל חיברנו מדפסת חדשה – הקרנל קורא למודול של המדפסת ומוסיף אותו לתוך עצמו בזמן אמת.

Virtual Machines

מדובר בתוכנית שתדמה מערכת הפעלה שיושבת מעל החומרה שלנו – למרות שהיא עצמה ישבה מעל מערכת הפעלה. הרעיון הוא, שבעת שנשתמש ב-Virtual machines נקבל ממשק שיושב מעל החומרה מתחת, ואז נוכל לכאורה להתקין מערכת הפעלה נוספת במחשב שלי.

בתמונה: דוגמה ל-virtual machine, מקבלים הפרדה מלאה.

- אנחנו רגילים לחשוב על שימוש בזה על מנת לקבל מערכת הפעלה אחרת במחשב שלנו. אך, ייתכן שנשתמש גם במחשב לינוקס ובכל זאת נוריד מכונה וירטואלית. מדוע? על מנת ליצור הפרדה בין המשתמשים – כל אחד יקבל מעין שרת משלו באמצעות המכונה הווירטואלית.



יתרונות שימוש:

- אפשר לבדוק את אותה אפליקציה שאנחנו מפתחים על אותה מערכת הפעלה מבלי שנצטרך לקנות מחשבים נוספים – לא צריך עוד חומרה. גם לא צריך להעלות מחדש את המחשב (שהרי יכלנו לבצע dual-boot ולהעלות למחשב כל פעם מערכת הפעלה אחרת).
- הפרדה מלאה בין כמה תהליכים שרצים.
- מונע פגיעה במערכת ההפעלה האמיתית – אם היה לנו תהליך שבגלל משהו זדוני שעשה הקריס לי את מערכת ההפעלה, עכשיו הוא לא יכול לעשות את זה. הוא במקסימום יפגע במכונה הווירטואלית ויגרום לי להעלות אותו מחדש.
- אפשר להריץ אפליקציות שונות שפותחו במקור למערכות הפעלה שונות ממה שיש לי על המחשב – מבלי לבצע ריסטארט של המחשב.

חסרונות שימוש:

- מייצרים עוד "שכבה" שצריך לעבור בדרך – וזה מאט את המהירות.
- לא נותן תאימות מלאה, תמיד ישנו מקום שלא נקבל את ההתנהגות המלאה של מערכת ההפעלה שנכתבה במקור.

סיכום מערכות הפעלה | © גיא יער-און

JVM: המכונה הווירטואלית של ג'אווה. class loader מקבל קבצי class משני מקורות – הקוד שכתבת וספריות המובנות של ג'אווה. הקבצים הללו מקבלים bytecode שזהו קוד שרק ה-JVM מבין ולא המעבד ישירות, הinterpreter לוקח את הbytecode ומעבדן אותו תוך כדי תנועה להוראות שמערכת ההפעלה יודעת להריץ. היתרון: בזכות המפרש לא צריך לשנות את הקוד שלנו.

System debugging

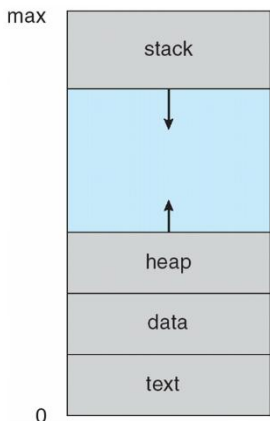
Log files אלו הראיות הראשונות, המערכת כותבת כל הזמן מעין יומן אירועים – אם משהו לא עובד אבל המחשב לא קרס, הולכים ללוגים לראות מה השתבש בדרך. ברמת האפליקציה (core dump) מדובר כבר בקריסה של תוכנה אחת, מערכת ההפעלה מצלמת את הזיכרון שהיה שייך לאותה תוכנה ושומרת לקובץ, כדי שהמפתח יוכל לראות מה היה בתוך המשתנים רגע לפני הפיצוץ. ברמת המערכת (crash dump) זהו המקרה החמור ביותר – כל מערכת ההפעלה קרסה, כיוון שהכל נפל המערכת שומרת צילום של הקרנל כדי שיהיה אפשר להבין איזה דרייבר גרם לכל המחשב להיעצר.

תהליכים – Process

בעבר – מערכות הפעלה אפשרו להריץ תהליך אחד (תוכנית אחת) בו זמנית. כיום: ניתן להריץ כמה בו זמנית.

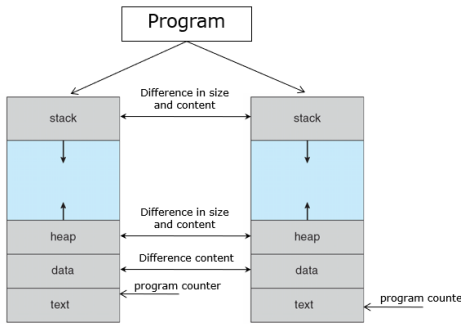
הערה: אנחנו נתייחס לjob processi כמושגים זהים בקורס חופפים זה לזה. בעוד בפועל – jobs מתייחס לbatch systems processi מתייחס למערכות מסוג time-shared (מציג גם את הדרך, לא רק תוצאה סופית).

- איזה תהליכים רצים במערכת? תהליכים של מערכת ההפעלה, ותהליכים של המשתמש שמכילים user code.
 - Program:** מדובר ביישום פאסיבית (היא לא משתנה) ששוכנת על הדיסק, בעוד **Process:** ישות אקטיבית אשר משנה באופן רצוף את מצבה.
 - הרצת התהליך מתבצעת בצורה סדרתית – זהו עקרון חשוב שכן בזכותו אנחנו תמיד יודעים איפה הפקודה הבאה שלנו.
 - התהליך יושב ב"מרחב זיכרון וירטואלי" – הוא יושב בצורה לוגית מכתובת 0 לוגית (הכוונה היא לכתובת מדומה) עד לכתובת max לוגית. שוב: אלו לא הכתובות האמיתיות, אך מה שרלוונטי לנו הוא המבנה שנשמר בזיכרון ביחד.
1. Text נשמר הקוד.
 2. Data נשמרים כל המשתנים הגלובליים – המשתנים שרצים עם התהליך לכל אורך חייו.
 3. Heap ישנן ההקצאות הדינמיות שעשינו.
 4. Stack נשמרים משתנים לוקליים (כל פעם שקוראים לפונקציה, שיחיו במשך חיי הפונקציה, הם נוצרים במחסנית).



סיכום מערכות הפעלה | © גיא יער-און

על בסיס אותה תוכנית, ניתן להריץ שני תהליכים או יותר במקביל שלכל אחד יש execution sequence משלו. למשל: כאשר פותחים כמה וכמה קבצי word, לכל אחד מהם יש sequence שונה. משמאל – דוגמה:



נבחין:

- הText יהיה זהה בשני הsequence אך הprogram counter יהיה שונה שכן יתכן שאנחנו נמצאים במקומות שונים בקוד.
- הdata יהיה זהה בגודלו, אך שונה בתוכנו (אותם משתנים, אך יתכן שערכיהם שונים).
- הheap, stack יהיו שונים זה מזה. (לא בהכרח, אך לרוב).

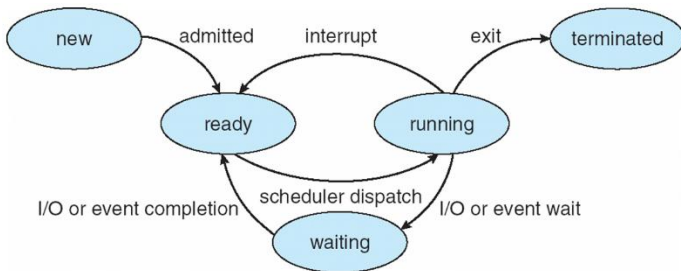
כל Process במהלך ריצתו משנה את המצב שלו (state). ישנם כמה מצבים:

- **New:** מדובר בתהליך שנמצא בתהליך יצירה, תהליך שעשינו לו loading. הוא עדיין "תינוק" ללא תעודת זהות, הוא רק מיוצר בשלב זה ולא ניתן להרצה.
- **Running:** כבר אפשר להריץ הוראות, אנחנו על המעבד.
- **Waiting:** מדובר במצב של המתנה, ממתינים לאירוע כלשהו שיקרה. למשל – IO של המשתמש שמעכב אותו.
- **Ready:** התהליך יכול להמשיך ולרוץ – אך אין לו כרגע את המעבד, לכן הוא ממתין.
- **Terminated:** התהליך קרס / הסתיים, וכעת מערכת ההפעלה צריכה לאסוף את המשאבים אחריו ולנקות אותו.

שאלה. כמה תהליכים יכולים להיות במערכת בכל רגע נתון בכל state?

תשובה. לא ניתן לדעת. הרי, בשביל שיהיו לנו כמה תהליכים בתוך new אנחנו זקוקים למס' המעבדים. לכן, הערך הזה קטן שווה ממס' המעבדים ואם נקצין: נאמר שזה 1 ומטה, שכן לא נרצה שיהיו לנו כמה תהליכים שנוצרים בזמנית. בדומה עבור Terminated ועבור running. כמה יהיה לנו waiting ובready? קטן שווה ממס' התהליכים במערכת.

כיצד תהליך עובר בין השלבים?

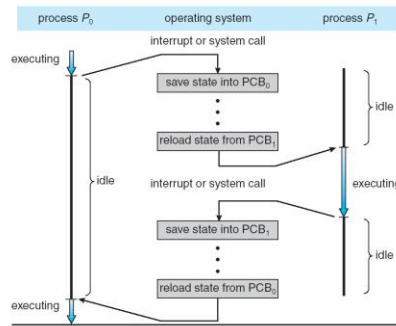


- מתחילים בnew, זה שלב ההיווצרות. בדומה: רק פעם אחת אנחנו terminated.
- משם, אנחנו עוברים אל מצב ready: תהליך זה נקרא **admitted** – נבחין שתהליך לא יכול לעבור ישירות אל running או waiting: שכן בשביל שיוכלו לבחור אותי בהרצה למצב running ויתנו לי את המעבד אנחנו צריכים להכנס למצב ready.
- ממצב ready ניתן לעבור למצב running אם הscheduler בחר בי להכנס (מיד נרחיב עליו). אנחנו למעשה יושבים ומחכים.
- ממצב running ניתן להגיע למצב terminated אם ביצענו exit, וניתן להגיע למצב waiting אם ביצענו IO למשל, וגם: ניתן לעבור למצב של ready – אם אנחנו למשל בtime shared והscheduler החליט שהיה לנו מספיק זמן – ואם למשל נכנס קוד של מערכת ההפעלה.
- ממצב של waiting ניתן להגיע אך ורק למצב ready: שכן חיכינו ועכשיו אנחנו מוכנים ומחכים לscheduler.

סיכום מערכות הפעלה | © גיא יער-און

process state
process number
program counter
registers
memory limits
list of open files
...

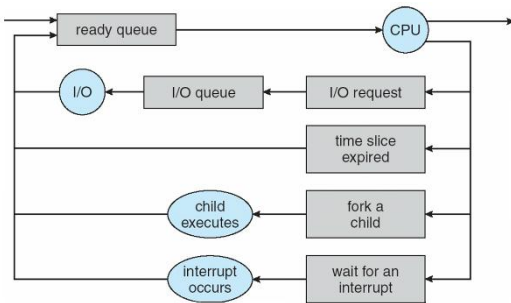
Process control Block (PCB): על מנת לתחזק את המעברים בין המצבים של התהליך, אנחנו שומרים מבנה נתונים בשם PCB – שם נשמור אינפורמציה כמו: מצב התהליך, מס' התהליך, PC (המקום בו אנחנו נמצאים בקוד), תוכן הרגיסטרים וכו'.



כיצד עוברים מתהליך לתהליך?

Process Scheduling Queues: המטרה שלנו היא שבכל עת יהיה תהליך שיכול לרוץ על המעבד (על מנת למקסם את ניצולת המעבד), לכן נשתמש ב-process scheduler: תורים בהם נעשה שימוש:

- Job queue: סט של כל התהליכים שבמערכת (לתור זה מגיעים כל התהליכים במערכת)
- Ready queue: סט של כל התהליכים אשר: 1. נמצאים בזכרון. 2. ממתנים להרצה.
- Device queue: סט של כל התהליכים שממתנים לIO.



במהלך הרצתו, התהליך עובר בין התורים השונים.

לדוגמה: תהליך נוצר ונכנס לתור ready. הוא יושב ומחכה – עד שהגיע תורו. הוא מבקש IO באמצעות system calls ונכנס לתור של IO, כשהIO שלו הסתיים הוא חוזר לתור של ready. מה עוד קורה לו בהמשך? Time slice expired: כשאנחנו חולקים את הזמן במעבד, נגמר הזמן שהוקצה לו, והוא חוזר שוב למצב של ready. יתכן ובהמשך – התהליך ייצור תהליך בן, ונרצה לבצע wait() ולעצור עד שהבן שלנו יסתיים, כעת נכנס למצב waiting. אפשרות נוספת – שאנחנו מחכים לinterrupt ולכן נמצאים בsleep.

Scheduler: ישנם שני schedulers עם מטרות שונות:

- **Long term scheduler:** בוחר התהליכים שיהיו ב-ready queue – בוחר מי יהיו בזיכרון. ה"יותר חכם", כיוון שאנחנו מפעילים אותו בתדירות יותר נמוכה ביחס ל-short-term, הוא "חושב קדימה" וההחלטות שלו מאוד משמעותיות והרות גורל. כל החלטה שלו – מעבירה תהליך מדיסק לזיכרון, לכן מופעל בתדירות די נמוכה. הוא שולט על מס' התהליכים שנמצאים בזיכרון בכל רגע (מושג זה נקרא **degree of multiprogramming** – דודי אמר לזכור מושג זה למבחן).
- **Short-Term scheduler:** בוחר את התהליך שירוצו על המעבד (מבין אלו שנמצאים ב-ready queue). מופעל כל כמה מילישניות, חייב להיות מאוד מהיר אחרת יהיה לנו overhead (תקורה – לא נרוויח ממנו כלום – אך יתפוס לנו משאבים בזמן שלא הרצנו כלום). לכן ישתמש בשיטות די פשוטות להחלטתו כי הוא חייב לקבל החלטות בצורה די מהירה.
- **Overhead:** תקורה. מתייחס לכל משאב של המחשב (זמן מעבד, זיכרון וכו') שנצטרך לא כדי לבצע את המשימה עצמה אלא כדי לנהל אותה.

תהליכים מאופיינים כאחד מהשניים הבאים לרוב:

- **IO bound**: תהליך שרוב הזמן שלו הוא מבלה את זמנו בפעולות של IO – רוב הקוד שלו מריץ הוראות של IO. למשל: גלישה ברשת (הרי מדובר בהרבה הדפסה/קבלה מהמסך וכו'. החישובים קצרים) או העתקה של קבצים גדולים.
- **CPU bound**: תהליכים שרוב הזמן שלהם מריצים הוראות חישוביות – למשל חישוב של π מבצע הרבה חישובים – ואין שם יותר מדי הוראות IO.

חשוב: ה Long term scheduler צריך לדאוג שבכל רגע נתון יהיה בזיכרון שילוב של תהליכי IO, CPU. מדוע? כי אם יהיה לנו בזיכרון הרבה תהליכי IO התורים של IO יתפוצצו – והמעבד לא יהיה בשימוש, והנצילות תרד. מצד שני, אם יהיה לנו בזיכרון הרבה תהליכי CPU תור ready יתפוצץ ולא יהיה נצילות ושימוש ב IO.

Context Switch

הגדרה: כאשר המעבד עובר לטפל בתהליך אחר, המערכת חייבת לשמור את state של התהליך המסוים ו"לטעון" את state השמור של התהליך החדש – תהליך זה נקרא context switch.

- context switch עצמו שמור ב PCB (מבנה נתונים לתחזוק המצב הנוכחי).
 - זמן ביצוע context switch הוא overhead שכן המערכת איננה מבצעת עבודה שמשמשת את המשתמש בזמן זה – אלא משמשת את עצמה בהמשך. משך זמן הביצוע של context switch מושפע מהתמיכה בחומרה.
- שאלה לדוגמה (ממבחן).** תהי מערכת מחשב עם N תהליכים שחולקים CPU באמצעות מנגנון בשם RR (כל תהליך מקבל אותו זמן CPU). נניח כי כל context switch אורך S מילישניות וכל time quantum אורך Q מילישניות. מהו הערך המקסימלי של Q כך שהזמן המקסימלי בין זוג instructions של אותו תהליך הוא T מילישניות?

פתרון: למעשה אנחנו נשאלים כמה זמן (חסום ב T) עובר בין זמן של תהליך x לפעם הבאה שירוצץ במעבד. נבחין כי בין התהליכים ישנם:

- N פעמים context switch: בזמן כולל של SN
- N-1 פעמים שיתבצעו תהליכים אחרים: בזמן כולל של (N-1)Q

$$\text{לכן נדרוש: } SN + (N - 1)Q < T \text{ ונקבל: } Q < \frac{T - NS}{N - 1}$$

Process creation: כיצד יוצרים תהליך? מישור צריך להריץ פקודה בשם create על מנת להריץ פרויקט. לכן – לכל תהליך יהיה תהליך סבא שידאג ליצור אותו. למעשה אנחנו נקבל מעין עץ תהליכים, כאשר ראש העץ הוא scheduler שמנהל את התהליכים (אותו יקצה boot program). כל תהליך צריך לקבל משאבים/זיכרון וכו'. מי ייתן לו אותם? ישנם כמה גישות:

- האבא נותן לתהליך הבן את המשאבים שלו – שיחלקו אותם יחדיו.
- האבא מקצה חלק מהמשאבים שלו שיוקצו עבור הבן.
- האבא והבן לא יחלקו משאבים – לבן יהיה משאבים משלו (זה מסוכן לנו: שכן ייתכן שיהיה תהליך שייצור בכוונה הרבה תהליכים בנים על מנת לקבל עוד משאבים עבורו).

הרצאה 4

כפי שאמרנו בסיום ההרצאה הקודמת, תהליך נוצר ע"י מישור אחר. בלינוקס – זה ע"י פעולת `fork()` והBoot program יוצר את התהליך הראשון. ומה לגבי ההרצאה עצמה? תהליך האב יכול לרוץ במקביל לתהליך הבן – בזכות multiprocessing. אך לא תמיד הם ירצו לרוץ במקביל: למשל, אם נרצה ליצור תהליך בן שישאל את המשתמש כל מיני שאלות, עד שהמשתמש לא יענה – נרצה שבזמן הזה תהליך האב יחכה. בשביל זה ישנה פעולת `wait()`.

- מרחב הזיכרון (memory space) של הבן תהיה זהה לתמונת הזיכרון של האב בעת שנוצר תהליך הבן, אך הם יהיו נפרדים.
- ניתן גם לטעון תוכנית חדשה לחלוטין לתוך תהליך הבן, באמצעות פעולת `exec()`.
- כיצד מבדילים בין האב לבן? באמצעות ערך החזרה של פעולת `fork()`. ערך זה הוא `pid`. אם `pid < 0` אזי תהליך יצירת הבן נכשל. אם `pid = 0` אזי מדובר בתהליך הבן. אחרת, `pid > 0` ומדובר בתהליך האב.

Process termination

בסיום פעולתו של כל תהליך הוא נדרש להריץ את הפקודה `exit()` – מדובר בsystem call. במצב זה מערכת ההפעלה:

- מוחקת אותו מרשימת התהליכים הקיימים (job queue)
- כל המשאבים שהוקצו לתהליך יוחזרו למערכת ההפעלה, והיא תרשום שהמשאבים הללו אינם מוקצים לאף תהליך.
- אם התהליך הנ"ל היה תהליך בן, אזי אם הייתה בקשת `wait()` מהאב יוחזר לו כי סיימו.

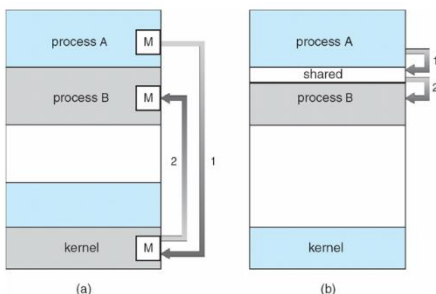
כמו כן, תהליך אב יכול לגרום להפסקת פעולת תהליך הבן באמצעות `abort`. מתי נרצה לעשות זאת?

- הבן משתמש ביותר מדי משאבים ממה שתכננו שהוא השתמש.
- המשימה שבשבילה נוצר הבן – כבר לא רלוונטית עוד. למשל: אנחנו סרבר, ויצרנו תהליך עבור בקשה מסוימת שהפכה ללא רלוונטית עוד. אז נרצה לבטל את התהליך הנ"ל.

Cascading termination: יש מערכות הפעלה שבעת שאבא ביצע `exit()` הורגות את כל העץ תחתיו – כל הבנים.

Interprocess Communication

תהליכים צריכים לשותף ולמצוא דרך לחלוק מידע בניהם. בין אם מדובר בתהליכים שרצים באותה מערכת או במערכות שונות. התמיכה בשיתוף המידע היא באמצעות IPC - interprocess communication. ישנם שני מודלים של IPC:



- **Shared memory:** מוקצה אזור בזיכרון הפיזי ששני התהליכים (או יותר) יכולים לגשת אליו. התהליכים קוראים וכותבים לאזור הזה כאילו הוא חלק ממרחב הכתובות הפרטי שלהם. זהו המנגנון המהיר ביותר כי הוא לא דורש התערבות של הליבה (Kernel) לאחר ההקמה, אך הוא דורש מהמתכנת לנהל סנכרון (כדי למנוע מצב ששני תהליכים כותבים בו-זמנית). בתמונה: מימין.
- **Message passing:** מערכת ההפעלה בונה מנגנון שמעביר "הודעות" בין התהליכים. היתרון שלו הוא שאין שום צורך לנהל דברים – זה תחום אחריותה של מערכת ההפעלה. החיסרון שלו הוא שהוא איטי יותר כי בגלל המעברים לkernel בכל הודעה הוא נדרש לעצור ולחכות. בתמונה: משמאל.

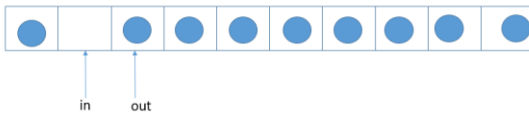
Producer-Consumer Problem: פרדיגמת היצרן-צרכן. מושג שנשתמש בו הרבה במערכות הפעלה. מודל קלאסי לשיתוף פעולה בין תהליכים, שבו תהליך אחד מייצר מידע ותהליך אחר משתמש בו. דוגמה נפוצה היא קומפיילר שמייצר קוד Assembly, אשר נצרך לאחר

מכאן על ידי ה-Assembler. כאשר משתמשים בפתרון של Shared Memory (זיכרון משותף) למימוש הפרדיגמה הזו, נהוג לדבר על שני סוגי חוצצים (Buffers) המשמשים להעברת המידע:

1. **Unbounded-Buffer (חוצץ ללא הגבלה):** במודל זה, אין הגבלה מעשית על גודל החוצץ (המקום אליו הם כותבים על מנת לתקשר בניהם) ולכן מבחינתם הוא אינסופי.

2. **Bounded-Buffer (חוצץ מוגבל):** מודל זה מניח שיש לחוצץ גודל קבוע ומוגדר מראש.

כיצד מממשים זאת? באמצעות שני מצביעים - in, out. במעין צורה ציקלית מעגלית. in הוא פוינטר שמצביע על התא הריק הבא שבו היצרן יכניס מידע חדש, out הוא התא שבו נמצא המידע הבא שהצרכן צריך לקרוא. ישנם שני מקרים:



- $in == out$: הצרכן יודע שאין לו מה לקרוא והוא צריך להמתין.
- $Out = (Buffer\ size) \% (in + 1)$: היצרן מתקדם ומגיע למצב שבו התא הבא שלו הוא out, המחסן מלא ולכן הוא צריך לחכות שהיצרן יוציא לפחות "תא אחד" ויתפנה מקום.

Message passing

- יוצרים משתנים משותפים ששני התהליכים מחזיקים.
- IFC Facility: מספק שתי פעולות, `send(message)` ו-`receive(message)`. אם זוג תהליכים רוצים לתקשר בניהם עליהם ליצור communication link בניהם ולהעביר הודעות באמצעות הפקודות. מימוש ה-communication link הוא באמצעות חומרה.
- את תהליך ההודעות ניתן לחלק לשני אפשרויות: direct – בצורה ישירה. indirect – התהליכים לא שולחים הודעות אחד לשני אלא משתמשים במתווך שנקרא Mailbox (תיבת דואר). כלומר – במקום שתהליך א' יצטרך להכיר את השם של תהליך ב' – שניהם צריכים בסך הכל להכיר את תיבת הדואר. מערכת ההפעלה צריכה לספק פונקציות כדי לנהל את התיבות הללו כמו יצירה של תיבה, שליחה וקבלת הודעות והריסת תיבה.
- השוואה בין תקשורת ישירה לעקיפה:

Interprocess Communication –
Message Passing (cont.)

	Indirect Communication	Direct Communication
method	Primitives are defined as: <code>send(A, message)</code> – send a message to mailbox A <code>receive(A, message)</code> – receive a message from mailbox A	Processes must name each other explicitly: <code>send(P, message)</code> – send a message to process <code>receive(Q, message)</code> – receive a message from process Q
Link established	only if processes share a common mailbox	established automatically
Link vs. process	link may be associated with many processes	link is associated with exactly one pair of communicating processes
Process pair vs. link	Each pair of processes may share several communication links	Between each pair there exists exactly one link
direction	Link may be unidirectional or bi-directional	The link may be unidirectional, but is usually bi-directional

Synchronization

המנגנון של message passing יכול להיות מאופיין כblocking or non-blocking: כלומר, האם השולח חוסם / לא חוסם את עצמו עד שההודעה מתקבלת. תצורת blocking נחשבת סינכרונית.

Blocking send: השולח מבצע block לעצמו עד שההודעה מתקבלת בצד השני.

Blocking receive: המקבל מבצע block לעצמו עד שההודעה זמינה לקבלה.

Non-blocking send: השולח שולח את ההודעה וממשיך בפעולתו – ללא חשיבות אם הצד השני קיבל אותה או שלא.

Non-blocking receive: המקבל מקבל את ההודעה גם אם היא invalid, הוא לא יושב ומחכה להודעה.

Buffering: המקום בו אנחנו משאירים את ההודעות ומחכים שיגיעו לצד השני. כמות המקום (ההודעות) שניתן לאחסן בו משתנה ויש כמה אפשרויות -

סיכום מערכות הפעלה | © גיא יער-און

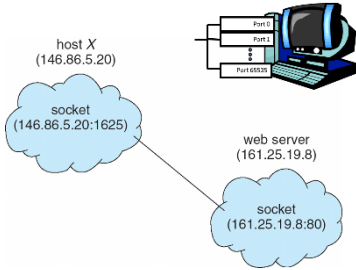
Zero capacity: לבאפר אין מקום להודעות שישבו ויחכו – בעת שליחת הודעה הצד השני צריך למשוך אותה, השולח חייב לבצע block לעצמו עד שהצד השני יקבל את ההודעה, הוא מחכה למקבל, המקבל מושך את ההודעה.

Bounded capacity: יש כמות סופית של הודעות שיכולות לשבת ולחכות, וכשהיה מלא אי אפשר לשלוח יותר הודעות.

Unbounded capacity: המקום בבאפר לא מוגבל – תמיד אפשר לשלוח הודעות. המצב הכי טוב לכאורה – מצריך הכי פחות ניהול.

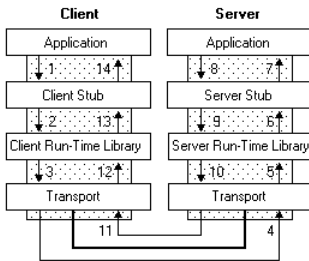
Socket – לא למבחן (לפי דודי)

עד כה, דיברנו על תקשורת בין תהליכים שהיו באותה מערכת הפעלה (אותו מחשב). ומה עם תקשורת בין תהליכים שנמצאים במחשבים שונים?



- לכל מכונה ברשת יש כתובת IP: X.X.X.X, שבאמצעותה ניתן לזהות את המחשב עצמו.
- זה לא מספיק לזהות את המחשב עצמו – לשם כך יש **port**: ערך מספרי שמייצג את האפליקציה בתוך המחשב. לצורך המחשה: יש בניין דירות, שמיוצג ע"י אותה כתובת, אך ה**port** הוא מספרי הדירה.
- בעת שנשייך את **port** ל**IP** ניצור **socket**. בעת ששני התהליכים מעוניינים לדבר – הם יצרו **connection**.
- ישנם שני פרוטוקולים שתפקידם להעביר הודעות בין אפליקציות שרצות על מחשבים שונים ברשת.
 1. UDP: "שולחים", לא אכפת לי מה יקרה" – טוב לשידורים חיים וכו'. לא מובטח שההודעה תשלח.
 2. TCP: אמין, מובטח שיגיע לנו הודעה.
- במהלך השנים התפתחו מוסכמות ל**port** מסוימים. למשל הוחלט כי למטרה מסוימת יוקצה **port** מסוים. ואז בשביל לבצע את אותה מטרה כל מה שצריך לדעת הוא את ה**IP**. למשל **PORT 80:HTTP**.

RPC – Remote procedure calls: לא למבחן (לפי דודי)



- מנגנון ה-RPC (Remote Procedure Call) נועד לגשר על הפער שנוצר כשעובדים עם **Sockets** – שבהם צריך לנהל דינאית את פתיחת הקשר, אריזת הנתונים (**Serialization**) ופענוח ההודעות. ה-RPC גורם לקריאה לפונקציה שנמצאת במחשב מרוחק להיראות ולנהל לקוד שלך בדיוק כמו קריאה לפונקציה מקומית. הוא "מסתיר" את כל הבלאגן של הרשת (**Sockets**, אריזת נתונים ושליחה) מאחורי קוד מתווך שנקרא **Stub**.
- התוכנה שלנו קוראת לפונקציה רגילה, קוד מקומי דרך תיווך (**stub**) אורז את הפרמטרים, תופס את הקריאה ושולח אותם לשרת. בצד השני: ה**server stub** פורק את החבילה, מפעיל את הפונקציה האמיתית בשרת ומחזיר את התוצאה באותה דרך.

RMI (Remote Method Invocation) הוא הגרסה של Java לעולם ה-RPC, והוא מאפשר לאובייקט שרץ ב-JVM (מכונה וירטואלית של ג'אווה) אחת להפעיל מתודות של אובייקט שנמצא ב-JVM אחרת, אפילו על מחשב שונה.

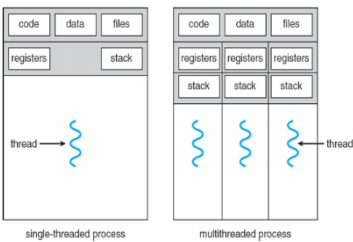
- **Busy waiting:** לרוץ בלולאה אינסופית ולבדוק תנאי – מבלי להמשיך. כאשר אנחנו במערכת עם מעבד יחיד – ביצוע **busy waiting** הוא חסר הגיון ואף מזיק. במערכת בעלת מעבד יחיד רק מעבד אחד יכול לרוץ בכל רגע נתון. אם תהליך א' מבצע **busy waiting** הוא תופס את המעבד ב-100%. הבעיה היא – תהליך א' מחכה לו יכול להשתנות רק ע"י תהליך ב' אבל כל עוד תהליך א' תקוע על המעבד בבדיקה שלו, תהליך ב' לא יכול לרוץ ולבצע את הפעולה שתשחרר את החסימה. המערכת תצטרך לחכות עד שיגמר זמן הריצה של א', ותבצע קונטקסט סוויץ' כדי שתהליך ב' יוכל לעבוד. בזמן הזה המעבד שרף זמן יקר.

Threads

עד כה הנחנו שכל תהליך מריץ תוכנית כך שבכל רגע נתון מתבצעת פקודה אחת בלבד של התהליך, אך ברוב האפליקציות המודרניות אנחנו רואים שימוש בmultithreading (שימוש בכמה חוטים בו זמנית, וכך מתבצעות כמה פקודות בכל רגע נתון).

לדוגמה: קליינט שולח בקשה לסרבר שמאזין לו, ובעת שמגיעה הבקשה הסרבר מייצר thread חדש שייתן את השירות לאותו קליינט וישוב להאזין לport שהיה בו. נשאל את השאלה: למה שלא ייצור תהליך חדש? מיד נבין זאת.

- threads רצים במסגרת התהליך והאפליקציה הבודדת.
- הרבה מהמשימות של אותו תהליך לרוב יכולים להיות ממומשים ע"י threads שונים. למשל: בעת שימוש בword נרצה שיהיה thread נפרד לטיפול בתצוגה, אחר לטקסט עצמו, תגובה להקשה על מקשים, הבאת מידע/שמירת מידע, וכו'.
- kernel של מערכת ההפעלה ממומש גם כן כמולטי-טרד.



הthread מורכב מ: thread id, program counter, register set, stack. עם זאת, thread חולק עם שאר threads של אותו תהליך את:

- Code section
- Data section
- משאבים אחרים של מערכת ההפעלה כמו קבצים וכו'.

יתרונות של threads:

אז למה שלא ליצור עוד תהליך במקום תרד?

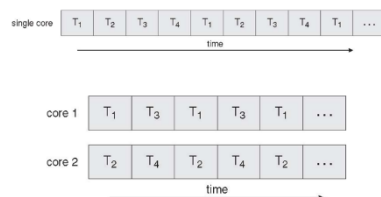
- הרבה יותר כלכלי ויעיל לבצע context switch לthread ולא לprocess שכן הם חולקים את משאבי התהליך. מדובר על פי 5 זמן בערך. כמו כן, הרבה יותר יעיל (פי 30) ליצור thread מאשר תהליך.
- מאפשר לתוכנית להמשיך לרוץ גם כאשר חלקים ממנה הם block (למשל יצאנו לIO) או שמבצע פעולה ארוכה (שהרי תרד רץ על חלקים שונים בתוכנית).
- threads חולקים את אותו הזיכרון והמשאבים של התהליך.
- אותו תהליך יכול לרוץ על מס' מעבדים במקביל באמצעות שימוש בthreads (כל אחד על מעבד שונה).

Multicore Programming

מערכות שהן multicore מביאות איתן אתגרים חדשים:

- קשה למצוא תחומים/ פעילויות שניתן לחלקם בצורה נקיה למשימות שיכולות להתקדם במקביל.
- חלוקת data ויצירת מבני נתונים מתאימים לתמיכה בהרצה במקביל.
- קושי משמעותי בtesting וdebugging. שכן הקוד הפך מדטרמיניסטי – לקוד multithread שכבר לא דטרמיניסטי – יתכן שהבאג קרה כי תרד אחד התקדם מהר מדי מתרד אחר: וכשנריץ שוב יתכן שזה כבר לא יקרה.

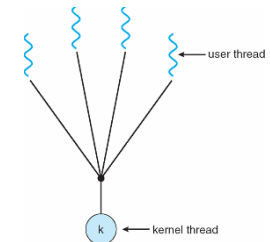
כיצד זה נראה בפועל? משמאל בתמונה. יהיו כמה ליבות (cores) שיעשו time-sharing. כשיש single core זו לא באמת ריצה במקביל לעומת המקרה שלמטה – שם זה ממש ריצה במקביל.



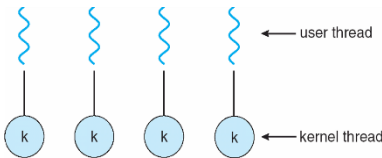
סיכום מערכות הפעלה | © גיא יער-און

Kernel threads: התרדים שמכירה מערכת ההפעלה. כלומר, threads שמכיר scheduler. כשהמתזמן מחליט מי ירוץ עכשיו, הוא בוחר ישירות קרנל תרד.

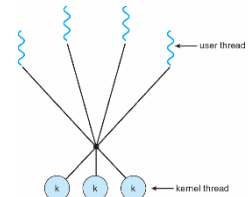
User threads: התרדים שמנוהלים באמצעות ספרייה (כמו pthread) בשם threading library ומשובצים ומנוהלים ע"י הספרייה עצמה. היא זו שמחליטה איזו תרדים כרגע רצים. מערכת ההפעלה לא יודעת שהם קיימים – היא רואה רק תהליך אחד גדול. הספרייה היא זו שמנהלת "לוח זמנים פנימי" ומחליטה מי מבניהם ירוץ עכשיו בתוך הזמן שהתהליך קיבל. היתרון הוא שהם מהירים מאוד, החיסרון הוא שאם תרד אחד מבצע פעולה חוסמת מערכת ההפעלה חוסמת את כל התהליך! כי היא לא מודעת באמת לכך שיש שם עוד תרדים בפנים. על מנת שuser threads באמת ירוצו על המעבד – הם חייבים "להתלבש" על קרנל תרד. ישנם כמה מודלים של מיפוי - המיפוי הוא הדרך שבה אנחנו מחברים בין העולם הווירטואלי של התוכנה לבין העולם הפיזי של המעבד. ישנם מס' מודלים:



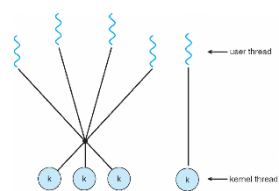
1. **Many-to-one:** מספר user-threads של התהליך ממופים כולם לkernel thread אחד. מה היתרון של הדבר הזה? זה מאוד קל, וכל הניהול של threads ינוהל בספרייה עצמה (זה יעיל מבחינת משאבים ותקורה). החיסרון הגדול באמת זה שאם אחד התרדים מבצע פעולה חוסמת מערכת ההפעלה חוסמת את כל התהליך, ואז גם אם יש תרדים אחרים שיכולים להשתמש במעבד כרגע הם לא יכולים לרוץ כרגע כי המערכת חסומה. כמו כן: אין כאן באמת הרצה במקביל, גם אם למעבד יש כמה ליבות.



2. **One-to-one:** לכל user-thread ישנן kernel-thread. נקבל מקביליות אמיתית בהרצה, אין בעיות של חסימה וכו'. אבל: כמובן שיש חסרונות – זה יקר מאוד! ויש גם overhead שגורם לניהול של הקרנל תרדס ומביא הרבה מגבלות. יהיו הרבה תהליכים שלא יוכלו לרוץ בפועל כי נגיע למגבלה על כמות התרדים המקסימלית.



3. **Many-to-many:** במקרה זה ישנם m kernel threads וישנם n user threads. ומתקיים $m \leq n$. גם כאן פתרנו את החסימה.



4. **Two-level model:** במקרה זה, לחלק מהuser threads שחשובים לנו אנחנו נעניק להם kernel thread ייחודי להם. ולשאר threads אנחנו נעבוד במודל many-to-many.

Java Threads

גם בjava ניתן ליצור threads. ניתן להרחיב את המחלקה Thread, או לממש ממשק בשם Runnable. המתודה start() מקצה זכרות ומאתחלת תרד חדש במחשב.

Threading Issues

יש המון בעיות שנוצרות מתרדים. נדון בכמה מהם:

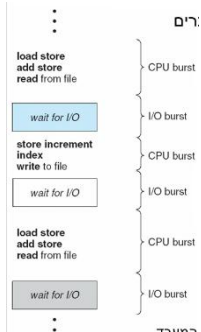
- נניח וביצענו fork(). הוא צריך לשכפל רק את thread הקורא (זה שיצר את התהליך החדש) או את כל התהליך? העיצוב הנכון הוא בהתאם למטרת היצירה. אם מיד לאחר fork נבצע exec הרי כדאי לשכפל רק את התרד הקורא (נחסוך במשאבים ובזמן, הרי במילא נדרוס הכל). אחרת, נשכפל את הכל.
- נרצה לפעמים לסיים פעולתו של thread לפני שהוא סיים את פעילותו. למשל – אם מס' תרדים מחפשים בבסיס נתונים מידע ואחד מהם מצא את המידע. נרצה להפסיק את האחרים. ישנן שתי גישות עיקריות לעצירה:
 1. גישה אסינכרונית: "להרוג" את התרד. לא בהכרח נצליח לשחרר את המשאבים כמו שצריך.
 2. Deferred cancellation: מאפשר לתרד המטרה לבדוק באופן תקופתי האם עליו להתבטל – באמצעות בדיקה של דגל flag.

סיכום מערכות הפעלה | © גיא יער-און

- סינגלים מועברים לתהליך ונעשה שימוש בהם על מנת להעביר לתהליכים מידע לגבי אירועים רלוונטיים (למשל: חלוקה באפס או גישה לכתובת לא חוקית). מה הבעיה? לאיזה תרד נעביר את הסינגל הנ"ל? אפשר להעביר לכל התרדים של התהליך (ואז תקורה), אפשר להעביר לסט של תרדים שנקבעו מראש, אפשר להשתמש בתרד ייעודי שאליו יועברו כל הסינגלים, ואפשר להעביר לתרד שהסינגל הכי רלוונטי אליו (אך זה מחייב הבנה של לוגיקה).
 - Threads pool: הרעיון הוא יצירה של מס' תרדים מראש שנמצאים במאגר ומחכים למשימות. יש לכך יתרונות כמובן:
1. מהירות – מהיר יותר לטפל בבקשה לרוב באמצעות תרד קיים מאשר ליצור תרד חדש מאפס
 2. מאפשר להגביל את מס' התרדים באפליקציה לגודל המאגר הקבוע מראש. הסיכון ללא המאגר הוא שכמות בלתי מוגבלת של תרדים עלולה לכלות את כל משאבי המערכת.

הרצאה 5

בשיעורים הקודמים אמרנו שישנם שני סוגים של schedulers. בפרט, short term scheduler קובע מי יהיה בכל רגע נתון על המעבד (וה Long קובע מי יכנס לready queue). נציג את הרעיון של CPU Scheduling שהוא הבסיס למultiprogramming.



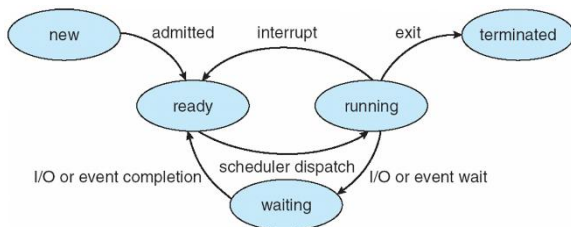
מחזור החיים של כל תהליך מורכב מרצף של מעברים בין הרצת דברים על המעבד, לבין ביצוע פעולות של IO, כאשר burst הוא רצף. כלומר הוא מחליף בין CPU burst to IO burst, ככה לסירוגין.

- תמיד מתחיל CPU burst ותמיד מסתיים בו. מדוע? כי הפקודה הראשונה והאחרונה שצריכים להריץ – תמיד רצים על גבי המעבד (מסתיים ב()exit שהרי מתבצעת על גבי המעבד).

בהינתן תהליך, ניתן לבנות היסטוגרמה של השכיחות היחסית של כל burst. ניתן לראות מכל היסטוגרמה כזו שבדרך כלל יש הרבה burst קצרים – ומעט מאוד burst ארוכים (אלו למען חישובים ארוכים לרוב, שלא קורים יותר מדי).

בהינתן ההבנה הזו, נבחין כי המעבד מתפנה יחסית די מהר – כל כמה מילישניות. אנחנו לא רוצים שהמעבד יהיה פנוי ולא יעשה כלום – זה יוריד את נצילות המעבד. לכן, נרצה בזמן זה (שהמעבד מתפנה) להכניס תהליך אחר לקבל זמן מעבד. כלומר – בעת שתהליך מסוים עבר לwaiting הוא יפנה את מקומו לתהליך אחר. מי שיחליט על התהליך הבא שיכנס – יהיה Short Term Scheduler.

מבין כל התהליכים שנמצאים בready queue המעבד יבחר מי כעת יקבל זמן מעבד.



מתי יש להפעיל את הCPU Scheduler?

- כאשר תהליך עובר ממצב running-> waiting
- כאשר תהליך עובר ממצב running->ready
- כאשר תהליך עובר ממצב waiting->ready (התהליך חיכה למשל לקריאה מתוך קובץ, קיבל interrupt, וכעת הוא מוכן. במצב זה נרצה לזמן לבדוק אם ניתן להריץ אותו).
- כאשר תהליך מסיים את חייו.

המצב הראשון והרביעי – שונים מהשני והשלישי. מדוע?

- במצב הראשון והרביעי – בעת שמתפנה המעבד, במצבים אלו נקרא לזה: nonpreemptive – זה אומר שהscheduler לא עוצר תהליכים באמצע על גבי המעבד. כלומר: רק כאשר התהליך עצמו מחליט לעצור – scheduler יעצור אבל הוא לא יעצור תהליך באמצע. במצב זה – המעבד מתפנה כי לתהליך אין ברירה אלא לעצור (כי נגמר התהליך, או שיוצאים למצב המתנה!).
- במצבים השני והשלישי, מתבצע preemptive – המעבד יכול לעצור תהליך באמצע למרות שהוא יכול להמשיך לרוץ.

Dispatcher: הוא זה שמבצע את החלטות scheduler. הוא מעביר את השליטה לתהליך שנבחר – מבצע context switch, עובר לuser mode, טוען את הPC של התהליך החדש וממשיך משם את הריצה. הזמן שלוקח לו לבצע את ההחלפה (לעצור את התהליך שרץ ולהפעיל במקומו תהליך אחר) נקרא Dispatch latency.

קריטריוני scheduling:

- ניצולת המעבד – אחוז הזמן בו המעבד עסוק בהרצת פקודות (ולא במנוחה או המתנה). נשאף לכמה שיותר גבוה אך זה נדיר שנגיע למאה אחוז. לעיתים מדובר במדד בעייתי שכן לא מפריד בין זמן מעבד של מערכת ההפעלה לבין זה של המשתמש. **נרצה למקסמו.**
- Throughput (תפוקה): מספר התהליכים שמסיימים ריצה בפרק זמן נתון. **נרצה למקסמו.**
- Turnaround time: הזמן הכולל להרצה של התהליך, כולל את: הזמן שהתהליך המתין עד שנכנס לזיכרון (אשר אותו הכניס לזיכרון ה Long term scheduler), הזמן שהמתין בready queue – **זה היחיד ששולט עליו ה short term scheduler: לכן הוא הכי חשוב!** זמן ההרצה על המעבד, הזמן שבילה בביצוע (או בהמתנה) IO. **נרצה למזער.**
- Waiting time: הזמן היחיד שבאמת מושפע מהscheduling. **נרצה למזער.**

סיכום מערכות הפעלה | © גיא יער-און

- Response time: הזמן מהרגע שהתהליך ביקש את המעבד, עד לרגע הראשון שבו התהליך קיבל אותו לראשונה. (מדוע לראשונה? זה הזמן שבו יש "התקדמות" לתגובת המשתמש – זה נותן למשתמש את ההרגשה שהתהליך איתו, ולא תקוע). **נרצה למזער.**

מעבר לגישה הפשוטה של למקסם או למזער כל אחד מהמדדים – נרצה לעבוד בminmax: שיקולים של – לא שווה למזער את כולם ואחד המדדים יהיה מאוד גבוה. יתרה מזאת: נרצה למזער את השונות, ולא את הממוצע.

כעת נדון בשיטות Scheduling.

First-Served (FCFS) Scheduling

The simplest to implement!

Process	Burst Time
P_1	24
P_2	3
P_3	3

- Suppose that the processes arrive in the order: P_1, P_2, P_3
The Gantt Chart for the schedule is:



- Waiting time for $P_1 = 0$; $P_2 = 24$; $P_3 = 27$
- Average waiting time: $(0 + 24 + 27)/3 = 17$

השיטה הפשוטה אומרת את הדבר הבא: אסתכל על כל התהליכים שנמצאים בready queue, ואכניס ראשון את התהליך שהגיע ראשון, שני מי שהגיע שני וכו'. כלומר בהינתן תהליכים שהגיעו בסדר $P_1, P_2, P_3, \dots$ נריץ ראשונה את P_1 ואז P_2 וכו'.

ניצור את Gantt chart (תרשים זמן של התהליכים, כמו שמופיע בדוגמה משמאל):

נבחין כי אנחנו מזניחים בדוגמה משמאל את context switch בין כל החלפה של תהליכים.

נבחין ששיטה זו עובדת בnonprimitive – אין שום כוח בעולם שיוריד תהליך מהמעבד! בהכרח: בגלל שתהליך הגיע במיקום הו ל-ready queue גורר שהוא ירוץ במיקום הו. סה"כ נגדיר לבסוף:

$$\text{Average waiting time} = \frac{\text{waiting-time}(P_1) + \text{waiting-time}(P_2) + \dots + \text{waiting-time}(P_n)}{n}$$

ניצולת המעבד בשיטה זו היא 100%.

Suppose that the processes arrive in the order

P_2, P_3, P_1

- The Gantt chart for the schedule is:



- Waiting time for $P_1 = 6$; $P_2 = 0$; $P_3 = 3$
- Average waiting time: $(6 + 0 + 3)/3 = 3$
- Much better than previous case
- Convoy effect short process behind long process $\rightarrow$ lower device utilization
- FCFS is nonpreemptive

נבחין שלמרות שהתהליכים כפי שנאמר בשאלה הגיעו בזמן זהה – הסדר שלהם שונה: מדוע זה כך? ברוב המקרים אם שני תהליכים הגיעו בזמן $t=0$ המתזמן יכניס אותם לפי הסדר שבה מופיע בטבלת התהליכים שכתובה לו. ולכן יופיעו בסדר שונה. סיבה נוספת יכולה להיות קשורה למזהה התהליך – המתזמן יכניס את התהליך עם הPID הנמוך יותר.

הconvoy effect הוא ההשערה לפיה תהליך מאוד ארוך – ימשוך את כל התהליכים אחריו, ויגדיל כתוצאה מכך את הAverage waiting time. למשל, אם התהליכים (מהדוגמה קודם) היו מגיעים בסדר שונה היינו מקבלים:

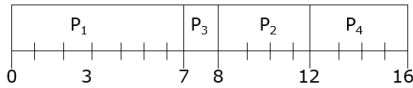
Shortest-Job-First (SJF) Scheduling

שיטה זו בוחרת בכל שלב להריץ את התהליך שהCPU-Burst שלו הוא הקטן ביותר. ישנן שתי סכמות למימוש:

- Nonpreemptive: ברגע שתהליך משובץ על המעבד, לא ניתן לעצור אותו עד שישלים את CPU burst הנוכחי שלו.
- Preemptive: אם מגיע תהליך חדש ברגע נתון עם CPU burst קצר יותר מהburst שנותר להרצה של התהליך הנוכחי, תבוצע החלפה לתהליך הקיים ובמקומו ירוץ התהליך החדש.

Process	Arrival Time	Burst Time
P_1	0.0	7
P_2	2.0	4
P_3	4.0	1
P_4	5.0	4

□ SJF (non-preemptive)



□ Average waiting time = $(0 + 6 + 3 + 7)/4 = 4$

בתחילה בזמן 0 מופיע רק התהליך הראשון ולכן נכנס. הוא מתבצע כל הזמן שלו כי אנחנו ב nonpreemptive. לאחר מכן – בזמן 7 נמצאים במערכת שלושת התהליכים האחרים.

זה עם הזמן הקטן ביותר הוא תהליך 3 שמגיע תורו. לאחר מכן בהתאם – מופיעים תהליכים 2 ו 3. סה"כ הזמן הוא הממוצע של זמן ההמתנה.

החישובים לכל תהליך מתבצעים לפי הנוסחה מטה.

ואם נעבור ב preemptive נקבל:

Process	Arrival Time	Burst Time
P_1	0.0	7
P_2	2.0	4
P_3	4.0	1
P_4	5.0	4

□ SJF (preemptive)



□ Average waiting time = $(9 + 1 + 0 + 2)/4 = 3$

בתחילה תהליך אחד הוא היחיד שקיים, רץ למשך 2 שניות וכעת זמן burst שלו הוא 5. בשנייה 2 מגיע תהליך 2 שנכנס כי יש לו פחות, ובשנייה 4 נכנס תהליך 3 שמבצע את כל הזמן שלו. בשנייה 5 השליטה חוזרת לתהליך 2 וכן הלאה.

כיצד מחשבים לכל תהליך את זמן הריצה שלו? לפי:

$$waiting - time = end - start - burst_time$$

מדוע? זמן הסיום פחות זמן ההגעה זה משך הזמן שהתהליך בילה במערכת. מזה נחסיר את הזמן שהוא אכן עבד. סה"כ נקבל את זמן ההמתנה במערכת.

- **טענה:** שיטה זו מבטיחה את ה minimum average waiting time לכל סט נתון של תהליכים.
- **הוכחה:** נניח שיש תהליך ארוך לפני תהליך קצר, נחליף בניהם. התהליך הארוך שהיה לפניו קדימה בגודל התהליך הקטן (לכן הזמן של הקצר שקודם חיכה לארוך – הרוויח את גודל התהליך הארוך!). כיוון שארוך < קצר, הרווח תמיד גדול מההפסד.
- הוכחה זו נכונה כאשר context switch זניח! (שכן יתכן שזמן ההחלפה גדול). וכן: זה נכון כאשר מדובר ב preemptive.

מה הבעיה העיקרית בשיטה? איננו יודעים ויכולים להעריך את משך burst הבא. (אך ניתן לשאול תמיד את המשתמש שידוע נתון זה: אבל זה לא טוב כי הוא ישקר! הרי תמיד מישו יגיד "רק שאלה קצרה – אני נכנס ועוקף את התור", לכן לא נרצה לשאול את המשתמש). שהרי, המעבד לא יכול לדעת כמה זמן ייקח התהליך הבא שירוצ עליו!

במקום זה, אפשר לבצע חיזוי על פי תצפיות עבר. זה שכיח מאוד בעולם מדעי המחשב – וברמה בסיסית במערכות הפעלה. בבסיס השיטה עומד העיקרון הבא:

עקרון הלווקאליות: אם קרה משהו לאחרונה, סיכוי סביר שיקרה שוב משהו דומה בעתיד הקרוב.

Exponential Smoothing Methods: שיטת חיזוי לזמן burst. מספקים ממוצע משוקלל נע (עם משקלים שקטנים אקספוננציאלית) של כל תחזיות העבר. זה מתאים לדטה ללא מגמה של ירידה/ עליה. המטרה היא להעריך את הממוצע הנוכחי ולהשתמש בו כאמד לערך העתידי. באופן פורמלי:

$$F_{t+1} = F_t + \alpha(y_t - F_t)$$

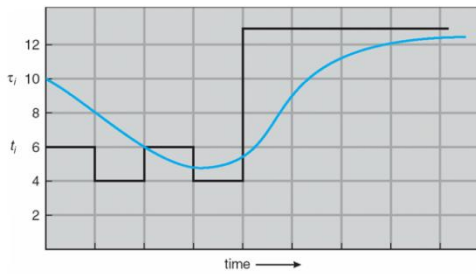
סיכום מערכות הפעלה | © גיא יער-און

- F_{t+1} : התחזית לזמן $t+1$.
- $Y_t(t)$: התחזית האחרונה שראינו בזמן t .
- a : ערך קבוע המקיים $0 < a < 1$.

מדוע מדברים על אקספוננציאלי? אם נפתח את הנוסחה - נקבל את הנוסחה הבאה:

$$F_{t+1} = \alpha y_t + \alpha(1-\alpha)y_{t-1} + \alpha(1-\alpha)^2 y_{t-2} + \alpha(1-\alpha)^3 y_{t-3} + \dots + \alpha(1-\alpha)^{t-1} y_1$$

הירידה האקספוננציאלית באה לידי ביטוי בחזקה של $(1-a)$: ככל שמתרחקים מהburst הנוכחי החזקה רק עולה. כיוון ש $1 > (1-a)$ זו ירידה אקספוננציאלית.



לדוגמה: כאשר $a = 0.5$

CPU burst (t_i)	6	4	6	4	13	13	13	...
"guess" (τ_i)	10	8	6	6	5	9	11	12

- את הערך של a נקבע באמצעות חיזוי ערכים שונים של a (למשל $a=0.1, 0.2, \dots, 0.9$). הערך של a שנותן את הroot-mean-square error הקטן ביותר הוא זה שנשתמש בו.

- כיצד נקבע את F_1 ? הרי זמן ההתחלה לא ידוע לנו. ניתן או לקחת את הערך הראשון עצמו בזמן, או ממוצע של כמה ערכים ראשונים בסדרה.

Priority Scheduling

- שיטה נוספת שבה לכל תהליך מצמידים מספר שמייצג את העדיפות שלו. המעבד מוקצה לתהליך עם העדיפות הגבוה ביותר (המספר הגבוה או הנמוך ביותר - אפשר גם ככה וגם ככה).
- גם בשיטה זו ניתן לבצע preemptive, nonpreemptive: אם יגיע תהליך עם עדיפות גבוהה יותר - נשנה את הסדר הרצה על המעבד (נפסיק נוכחי ונחליף).
- גם SJF הוא סוג של משיטה זו שמתעדף לפי burst time.
- מה הבעיה בשיטה זו? אם תהליך עם עדיפות מסוימת מחכה, ובכל שלב מגיע תהליך אחר שעוקף אותו. (למשל: עומדים בתור למשרד הפנים ובמשך שעות מגיעים כל הזמן אנשים עם פטור מתור). זה נקרא starvation. פתרון אפשרי לבעיה זו היא Aging: משפרים את העדיפות של התהליך ככל שהוא מחכה יותר זמן.

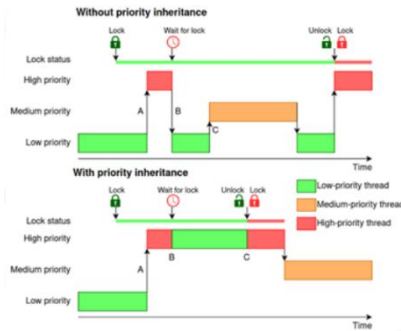
דוגמה: בתחילה ירוץ כאן תהליך 2 שהוא בעל העדיפות הגבוהה ביותר, לאחר מכן תהליך 5, תהליך 1, 3, 4 וכו'. זה כמובן בnonpreemptive.

Process	Burst Time	Priority (1=highest)
P_1	10	3
P_2	1	1
P_3	2	4
P_4	1	5
P_5	5	2

Priority scheduling Gantt Chart



Average waiting time = 8.2 msec



Priority Inversion (inheritance): היפוך עדיפויות. נניח מצב שבו תהליך פחות חשוב מצליח לעכב תהליך מאוד חשוב. זה נשמע לא הגיוני במערכת עם עדיפויות – אך קורה בגלל משאבים משותפים. נניח וישנם 3 תהליכים, L, M, ו-H. L נמוך, M בינוני, ו-H גבוה. מתבצע התרחיש הבא:

- L מתחיל לרוץ ותופס מנעול על משאב מסוים (קובץ נניח).
- H מתעורר, כיוון שיש לו עדיפות גבוהה הוא מעיף את L מהמעבד ומתחיל לרוץ.
- בשלב מסוים, H צריך את אותו משאב של L מחזיק. לכן H נחסם ומחכה של ישחרר את המנעול.

- עכשיו L חוזר לרוץ, מסיים, משחרר את המנעול. ואז: תהליך M מתעורר – כיוון ש-M חזק יותר מ-L הוא מעיף את L מהמעבד.
- התוצאה: תהליך H החשוב מחכה ל-L אבל L לא יכול לרוץ כי M תפס לו את המעבד – כלומר תהליך בינוני מעכב תהליך גבוה.

על מנת לפתור את זה משתמשים בירושת עדיפויות – בעת שתהליך H מנסה לתפוס מנעול שמוחזק ע"י L המערכת אומרת כי ניתן לזמנית את העדיפות של H. כעת L חסם מאוד – וכש-M יתעורר הוא לא יכול להעיף את L מהמעבד. אם במקרה M גדול מ-H, ירושת עדיפויות גם לא תעזור כי באמת M אמור לרוץ (הכי חזק כרגע).

Round Robin (RR)

- שיטה זו בעיקר למערכות מסוג time-sharing.
- כל תהליך מקבל זמן יחסית קצר (time quantum) על גבי המעבד. בד"כ בין 10-100 מילישניות. זה מתבצע בצורה ציקלית (מעגלית) על פני תור מסוים.
- שיטה זו מבטיחה כי אם ready queue ישנם n תהליכים והtime quantum הוא q אז כל תהליך יקבל $1/n$ מזמן המעבד באינטרוולים רצופים של q יחידות זמן ואף תהליך לא ימתין יותר מ-q(n-1) (כל קודמיו, כפול q הזמן שלוקח) יחידות זמן.
- אם q גדול: זה כמו FIFO.
- שיטת RR מבטיחה שאף תהליך לא ישאר "רעב" (starvation).

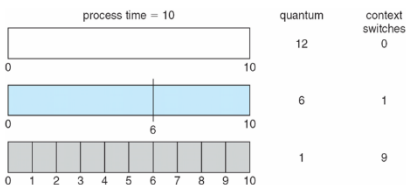
לדוגמה: אם $q=20$, שיטה זו תראה כך (כאשר המספרים שכתובים מעלה הם כמה זמן נותר לכל אחד מהתהליכים לרוץ). חוזרים בכל שלב בצורה ציקלית – לאלו שנותרו עוד בתור (remaining time > 0).

□ The Gantt chart is:

Process	Burst Time
P_1	53
P_2	17
P_3	68
P_4	24

53	33	33	33	13	13	13	0	0	0
17	17	0	0	0	0	0	0	0	0
68	68	68	48	48	28	28	28	8	0
24	24	24	4	4	4	0	0	0	0
P_1	P_2	P_3	P_4	P_1	P_3	P_4	P_1	P_3	P_3
0	20	37	57	77	97	117	121	134	154
0	20	37	57	77	97	117	121	134	154

□ Typically, higher average turnaround than SJF, but better response



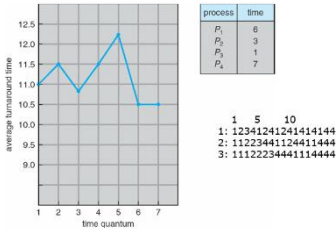
נתבונן משמאל – ישנה דילמה חשובה מאוד בבחירת זמן הtime quantum בשיטת RR: הקשר בין אורך הקוונטום לבין overhead (תקורה) של המערכת. נתבונן בתהליך שאורך 10 יחידות זמן.

1. מקרה ראשון: הקוונטום גדול מהזמן שהמעבד צריך. $10 < 12$. התוצאה היא שהתהליך נכנס למעבד ומסיים את כל העבודה שלו בפעם אחת. מספר context switch הוא 0 עבור אותו תהליך כתוצאה מכך. כלומר: ככל שהקוונטום גדול יותר, RR מתנהגת יותר ויותר כמו FCFS – מערכת יעילה ללא בזבז זמן על החלפות.
2. מקרה שני: קוונטום ששווה ל-6. קטן מ-10. התוצאה היא שהוא רץ 6 יחידות, מועף לסוף התור לכן מתבצע קונטקסט סוויץ' אחד ולכן שילמנו על זה קצת.

סיכום מערכות הפעלה | © גיא יער-און

3. מקרה אחרון: $q=1$. כעת יש 9 קונטקס סוויץ', יש overhead גבוה מאוד! המעבד יבלה יותר זמן בלהחליף מאשר לבצע עבודה בפועל.

הכלל שנגזר מדוגמה זו: נרצה ש $time_context_switch > q$.



עם זאת, כדאי שנשים לב לנתון הבא: זה לא תמיד נכון שאם נגדיל את q זמן השהייה הממוצע במערכת ישתפר. הקשר איננו לינארי! (למרות הדוגמה הקודמת). נתבונן בגרף משמאל שממחיש זאת. מדוע זה קורה? זמן השהייה הממוצע במערכת תלוי בשאלה "האם הקוונטום מאפשר לסיים תהליכים מהר?". בדוגמה משמאל, יש לנו תהליכים באורך 1 3 6 ו7. אם נבחר קוונטום של 6 – כולם פרט לאחרון יסיימו בסבב אחד. זמן השהייה הממוצע כן משתפר כאשר רוב התהליכים מסיימים את burst שלהם בתוך קוונטום אחד. אם הקוונטום קטן מדי ביחס לאורך התהליכים – יצטרכו הרבה קונטקס סוויץ'.

Multilevel Queue

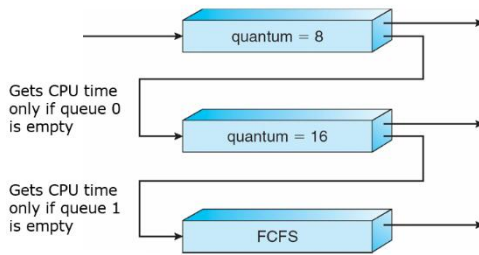
- חלוקת ready queue לכמה תורים נפרדים. למשל: תור עבור סוג תהליך. מדוע זה טוב? כי יש תהליכים שחשובים יותר, אותם ניתן להכניס לתור עבור סוגי תהליכים חשובים.
 - לכל תור נקבע אלגוריתם scheduling משלו (לא בהכרח זהים!).
 - כעת ישנן כמה גישות למי מהתורים יקבל את המעבד כרגע:
- עדיפות גבוהה – Fixed priority: שיטה היררכית נוקשה. תור Foreground (תהליכים אינטראקטיביים מול המשתמש) תמיד יקבל עדיפות – כל עוד יש שם תהליך אחד התורים האחרים לא יקבלו כלל את המעבד. ישנה כאן סכנה של starvation: אם המשתמש כל הזמן מזיז את העכבר או המקלדת, תהליכי הרקע לא ירוצו.
 - פריסת זמן – Time slice: גישה "דמוקרטית" יותר. המעבד מחלק את הכוח שלו לפי אחוזים. למשל מתוך כל שנייה של עבודה – 80% מהזמן לתהליכים בכוח החשוב, ו20% לחלק הפחות חשוב.

Multilevel Feedback Queue

זו השיטה הכי נפוצה במערכות הפעלה מודרניות. כאן, עובדים עם תורים נפרדים לפי מאפייני cpu burst שלהם. הרעיון המרכזי מתבסס על ענישה על שימוש מוגזם במעבד. השיטה הזו מתעדפת תהליכים קצרים ואינטראקטיביים על פני תהליכים כבדים. תהליך יכול לעבור בין מגוון תורים. תהליך תמיד נכנס לתור העליון – אם מסיים את עבודתו מהר, הוא נשאר בעדיפות גבוהה. אם הוא חמדן ומנצל את כל הקוונטום שהוקצה לו בתור הזה – הוא יורד קומה לתור נמוך יותר עם אלגוריתם אחר – אבל תמיד המערכת תדאג שאם יש מישהו בתור עליון, הוא יתבצע קודם. ישנה היררכיה. בשביל להשתמש באלגוריתם זה על המערכת לקבוע:

כמה תורים יהיו, איזה אלגוריתם רץ בכל תור, מתי מעבירים תור לתור נמוך יותר (לרוב כשמסיים קוונטום שלם), מתי מחזירים תהליך למעלה? (אם תהליך מחכה יותר מדי – לא נרצה שירעב, לכן לפעמים נעלה אותו למעלה), לאיזה תור נכנס תהליך כשהוא רק נוצר (לרוב להכי גבוה).

סיכום מערכות הפעלה | © גיא יער-און



לדוגמה: כל התהליכים נכנסים לתור הראשון, ש time quantum הוא בדיוק 8. אם התהליך צריך 8 או פחות יחידות זמן, הוא יסיים את חייו בתור הראשון. אחרת: אם burst שלו היה גדול ממש מ-8, הוא יכנס לתור הבא. קודם ירוצו כל התהליכים בתור הראשון. כשהוא יתרוקן – יתחילו לרוץ התהליכים בתור השני וכן הלאה. העדיפות של שיטה זו היא שהתהליכים הקצרים יחסית ירוצו מהר, ואם התהליך יחסית כבד, הוא כל פעם יתקדם קצת. בתור האחרון למשל ננקוט ב-FCFS.

קשר בין האלגוריתמים שראינו

- הקשר בין Priority to SJF: SJF הוא למעשה אלגוריתם של תזמון עדיפות לפי burst.
- הקשר בין FEEDBACK QUEUE ל-FCFS: הרמה הנמוכה ביותר של שיטת התורים היא FCFS, שכן תהליכים כבדים שוקעים לתורים התחתונים. בתור האחרון והנמוך ביותר, בדרך כלל כבר לא משתמשים ב-RR (אין טעם לקטוע תהליך שכבר הוכח לנו קודם שכבד מאוד) אלא נותנים להם לרוץ אחד אחרי השני לפי סדר ההגעה עד שסיימו בשביל לחסוך ב-overhead (שיגורם כתוצאה מהרבה קונטקס סוויץ').
- הקשר בין Priority to FCFS: FCFS הוא תזמון לפי סדר ההגעה.

Multiple-Processor Scheduling

כאשר ישנם כמה מעבדים Scheduling הופך להיות מסובך הרבה יותר. למעשה אנחנו צריכים לשאול את עצמנו אם אנחנו עדיין עובדים בתור אחד או כמה. ישנם שני מצבים:

- Asymmetric multiprocessing: רק מעבד אחד מנהל את התזמון בין התהליכים.
- symmetric multiprocessing: כל מעבד מתזמן לעצמו (כל המעבדים ניגשים לready queue משותף או שכל אחד מהם מנהל תור כזה משל עצמו – אבל אז זה פוגע ביעילות).

Processor affinity: זה צמידות. ישנו רצון שנהיה צמודים למעבד עליו אנחנו רצים כרגע. שכן, זה קשור ל-cache: תהליך שרץ על גבי מעבד ויעבור למעבד אחר עלול לאבד את cache שהוא בנה – ואז יצטרך מחדש לבנות cache שם וזה בזבז.

- Soft affinity – בקש ונשתדל לעשות.
- Hard affinity – התהליך דורש: לא מוכן לעבור למעבד אחר. תמיד דרישתו תענה.

Real-Time Scheduling

מערכות מסוג hard-real-time (כמו למשל "טיל כיפת ברזל") מאפשרות להשלים משימה קריטית בתוך פרק זמן מוגדר מראש. התהליך מבקש לקבל זמן מעבד עד לדדליין מסוים. scheduler מבטיח את קיום הבקשה – או דוחה אותה.

Algorithm Evaluation

נניח והגענו לעבודה חדשה – ואנחנו קיבלנו משימה: להחליט איזה אלגוריתם scheduling אנחנו נרץ. מה יהיו השלבים בדרך להחלטה?

1. לוקחים דוגמה קיימת – מחשבים מה יהיה המדד המתאים בכל אחד מהאלגוריתמים עבור אותה דוגמה, ולוקחים את זה עם המדד הטוב ביותר עבורנו.
2. תורת התורים – מחשבים בצורה הסתברותית את זמן ההגעה הצפוי (פואסונית וכו'). החיסרון של זה הוא שלרוב המודלים תורת התורים לא יכולה לפתור מתמטית.

Thread scheduling

כעת עוברים מרמת תהליך לרמת החוט. ישנן כמה שיטות לקביעה מי מהחוסים ירוץ על גבי המעבד.

1. Process contention Scope (PCS): מושג זה רלוונטי עבור Many-to-one and Many-to-Many. המנהל כאן הוא ספריית החוסים, נניח שיש תהליך אחד עם עשרה חוסים. אך מערכת ההפעלה הקצתה לו רק צינור אחד לקרנל - 10 החוסים הללו יריבו ביניהם למי הזכות להשתמש בצינור זה. התחרות פנימית בתוך התהליך!
2. System contention Scope (SCS): רלוונטי עבור חוסים שהקרנל מכיר. המנהל הוא המתזמן של מערכת ההפעלה. במודלים של one-to-one כל חוט שהמתכנת יוצר, מקבל חוט קרנל תואם. כאן החוט מתחרה מול כל שאר החוסים במחשב - התחרות ברמת המערכת כולה.

כעת נעבור לדון על scheduler של מערכות הפעלה ספציפיות.

1. Solaris scheduling: מערכת ההפעלה solaris - משתמשת בtime sharing (כלומר מסוג RR), על בסיס של multi-level feedback queue. המערכת קובעת חוק מעניין - ככל שהעדיפות גבוהה יותר, כך הקוונטום קטן יותר. מדוע? כי תהליכים בעדיפות גבוהה לרוב הם אינטראקטיביים שצריכים תגובה מיידית, אבל מסיימים את עבודתם מהר מאוד. תהליכים בעדיפות נמוכה לרוב הם CPU Bound (חישובים כבדים) שלא צריכים תגובה מהירה.
 2. Windows scheduling: המערכת של ווינדוס תמיד תעדיף להריץ את החוט בעל העדיפות הגבוהה ביותר. אם חוט חשוב יותר מתעורר - הוא קוטע את החוט שכרגע רץ. חוט מפסיק לרוץ באחד מהמקרים הבאים: הוא נעול, נגמר לו הקוונטום, או שחוט חזק יותר הגיע. ווינדוס מחלקת את העדיפויות ל-32 רמות.
 - רמות 1-15: רוב התהליכים הרגילים (כרום, ספוטיפיי וכו'). העדיפות שלהם גמישה - המערכת יכולה להעלות או להוריד אותם ברמה לפי צורך.
 - רמות 16-31: חוסים קריטיים של מערכת ההפעלה שלא עוברים שינויי עדיפויות. תמיד יקבלו עדיפות.
 - רמה 0: שמורה לidle Thread (מה שהמעבד מריץ כשאינו לו משהו אחר לעשות - חוט ה"בטלה" מלשון בטלון).
- לכל רמת עדיפות יש תור משלה, המתזמן סורק תמיד מהרמה הגבוהה ביותר (31) ומטה. רק כשתור גבוה ריק לגמרי, הוא ירד קומה.
3. Linux scheduling: יש עקרונות דומים למה שראינו בווינדוס. כמו כולם - משתמשת בpreemptive: היא לא מחכה שתהליך ירצה להסתיים לבד אלא עוצרת אותו אם מגיע אחד חשוב יותר. מחלקת את המשימות לטווחים של 0-140. אלו בטווח 0-99 הם העדיפויות הגבוהות ביותר, מיועדות למשימות קריטיות. האחרים - 100-140 הם התהליכים הרגילים. לתהליכים בעדיפות גבוהה יש קוונטום גבוה יותר, בשונה מsolaris.

עיבוד batch: שיטה בה המחשב מריץ קבוצת משימות ארוכות זו אחר זו ללא התערבות המשתמש. המטרה היא למקסם את ניצול המעבד. ממזערים את מס' הקונטקס סוויץ'.

סביבה אינטראקטיבית: מערכת בה המשתמש מתקשר עם המחשב בזמן אמת, והדגש העיקרי שלה הוא על זמן תגובה מהיר. יהיו החלפות בקצב תכוף.

CPU Bound: מצב בו קצב ההתקדמות של תהליך מוגבל ע"י מהירות המעבד בלבד (בחישובים מתמטיים כבדים למשל), כיוון שהוא כמעט ולא עוצר להמתנה של קלט פלט.

הרצאה 6

בהרצאה הקרובה נדבר על סינכרוניזציה.

מהי סינכרוניזציה?

- הבעיה שאנחנו מנסים להימנע ממנה היא data inconsistency: כלומר – אין עקביות בנתונים. מבצעים פעולות על הדטה, ומצפים להגיע לתוצאה מסוימת באופן עקבי אך לא מגיעים לתוצאה הזו. מתי עלולים להתקל בבעיה הזו? נראה כי אם התהליכים לא מופרעים במהלך ריצתם לא אמורה להיות בעיה.
- אם תהליך מאבד את המעבד כתוצאה מinterrupt מבלי שהשלים את burst הנוכחי שלו, אבל אף תהליך אחר לא יכול לגשת למשתנים של התהליך הנ"ל – שוב: לא אמורה להיות בעיה.
- מתי כן יש בעיה? אם התהליך מעדכן משתנים משותפים לתהליכים אחרים עלולה להיות בעיה כי אין תיאום וסינכרוניזציה ביניהם – הם פועלים על אותו זיכרון מבלי לדעת מה השני עושה.

Producer-Consumer Problem: כפי שראינו בעבר, יש לנו במודל זה

producer שאחראי על ייצור וconsumer שצורך ממנו. רצים על ה shared buffer בצורה ציקלית.

מה הבעיה והחיסרון במימוש הזה? אנחנו לא מנצלים את כל היחידות של

buffern אלא מנצלים n-1 תמיד בגלל in, out.

בואו ננסה לפתור את זה – ולהשתמש בכל הגודל של הבאפר: כיצד?

נוסיף משתנה בשם count, שיגיד לנו כמה איברים נכתבו ע"י producer ולא נקראו ע"י consumer. נתבונן על קוד שמממש את זה משמאל –

```
while (TRUE)
{
  /* produce an item and put
  in nextProduced */
  while (count == BUFFER_SIZE)
    ; // do nothing
  buffer[in] = nextProduced;
  in = (in + 1) % BUFFER_SIZE;
  count++;
}
producer

while (TRUE)
{
  while (count == 0)
    ; // do nothing
  nextConsumed = buffer[out];
  out = (out + 1) % BUFFER_SIZE;
  count--;
  /* consume the item in
  nextConsumed */
}
consumer
```

כל אחד משני התהליכים (צרכן, יצרן) מריץ קוד תקין. אבל: ההרצה של הקוד אצל היצרן והצרכן מתבצע במקביל – ועלולים להתקל בבעיה של data consistency עבור המשתנה count: מדוע? מכניסים את count הרי לתוך רגיסטר, ולרגיסטר עושים add\sub וכו'. ואז עלולים להתקל במצב הבא:

אם נניח כי count=5, נראה כי ייתכן ויקרה:

S0: producer execute	register1 = count	{ register1 = 5 }
S1: producer execute	register1 = register1 + 1	{ register1 = 6 }
S2: consumer execute	register2 = count	{ register2 = 5 }
S3: consumer execute	register2 = register2 - 1	{ register2 = 4 }
S4: producer execute	count = register1	{ counter = 6 }
S5: consumer execute	count = register2	{ counter = 4 }

מה קרה כאן? נניח שתהליך ראשון בשלב S₂ הפסיק והמעבד עבר לתהליך שני. מה שקרה הוא ששני התהליכים ניגשו בו זמנית למשתנה משותף count, ובשלב השלישי התהליך השני קיבל את הערך counter=5 ובמקום שהתהליך השני יקבל ערך counter=4 הוא קיבל 5. והמסקנה שלנו פשוטה: זה תלוי בסדר ההרצה – כיוון ששני התהליכים חולקים משאב משותף – משתנה.

סיכום מערכות הפעלה | © גיא יער-און

Race condition: "מצב מירוץ" – מצב שבו יש כמה תהליכים שניגשים ומשנים את אותו הדטה במקביל. התוצאה שנקבל תהיה תלויה בסדר הספציפי שבו מתבצעת הגישה של התהליכים השונים – ולכן התוצאה לא דטרמיניסטית. על מנת להימנע ממצב זה נרצה להבטיח שרק תהליך אחד יוכל לשנות את הדטה בו זמנית. נבחין: אם שניים רוצים לקרוא – אין לנו בעיה. אם שניים רוצים לכתוב – כן יש לנו בעיה. מה עם אחד שרוצה לכתוב ואחד לקרוא? לא נרצה לאפשר: שכן יתכן שזה שקורא יקרא ערך לא עדכני.

Critical Section Problem

כעת נגדיר מושג שיעזור לנו לפתור את ה-race condition. מדובר למעשה במקטע (סגמנט) של קוד שבו התהליך משנה משתנים משותפים עם תהליכים אחרים, טבלאות משותפות, כותב לקובץ וכו'. בהינתן מערכת עם n תהליכים $P_0, \dots, P_{n-1}$ שלכל אחד יש critical section משלו, נרצה שאם תהליך נמצא בcritical section שלו אז אף תהליך אחר לא יריץ פקודות באותו critical section. לכן הבעיה היא: לתכנן מעין פרוטוקול שיבטיח זאת. נרצה לפתח מנגנון שיבטיח שרק אחד מהתהליכים יריץ את הcritical section שלו.

```
do {
    entry section
    critical section
    exit section
    remainder section
} while (true);
```

כיצד נפתור את הבעיה? נוסיף קוד לתהליכים שיבטיח שרק תהליך אחד נכנס בו זמנית לcritical section.

נעטוף אותו בשני מקטעים:

- Entry section: צריך לרוץ לפני שנכנסים לקטע הקוד הקריטי.
- Exit section: צריך לרוץ מיד לאחר שסיימנו לבצע את קטע הקוד הקריטי.
- נבחין: זה לא קוד שעושה לנו לפונקציונליות של התהליך, אלא קוד שנועד לעזור לנו לפתור את בעיית הסינכרוניזציה. ולכן זה מעין overhead.

כשאנחנו מציעים פתרון באמצעות הוספת שני המקטעים הנ"ל – **נרצה שהפתרון יענה על שלוש דרישות מרכזיות:**

- אם תהליך P_i רץ כרגע בקטע הקוד הקריטי שלו, אז אף תהליך אחר לא יכול לרוץ בקטע הקוד הזה באותו זמן. (ברור).
- Progress: אם אף תהליך לא רץ כרגע בcritical section שלו, ויש תהליכים או תהליך שרוצים להכנס לקטע הקוד הקריטי שלהם – אז הבחירה בתהליך הבא שיכנס לcritical section שלו לא יכולה להידחות ללא הגבלה (כלומר, ישנה התקדמות בדרישותיהם). וכן, רק התהליכים שרוצים להכנס לקטע הקוד הקריטי שלהם – יהיו שותפים לתהליך הבחירה של מי יכנס לקטע הקוד הקריטי כרגע.
- Bounded waiting: חייב להיות חסם על מס' הפעמים שבהם תהליכים אחרים יכולים להיכנס לcritical section שלהם לאחר שתהליך מסוים ביקש להכנס לקטע הקוד הקריטי שלו, ולפני שהבקשה שלו מאושרת. (כלומר, מעין "פייריות" בין התהליכים. ישנו חסם על מס' הפעמים שאחרים יכנסו לקטע הקוד הקריטי שלהם מהרגע שאני ביקשתי להכנס – זה עוזר לנו להימנע ממצב של starvation).

Peterson's Solution

- הפתרון הראשון שנציג לבעיית הcritical section. מדובר בפתרון מבוסס קוד – בניגוד לאלו שנראה בהמשך שמבוססים על חומרה. אך: הפתרון מתאים לתרחיש של שני תהליכים (ההרחבה של יותר משני תהליכים – יכולה להיות במבחן דודי יותר מרמז על זה!): ההרחבה וההסבר נמצא במצגת 6 בשקופית האחרונה).
- הפתרון מניח שאי אפשר לעשות interrupt באמצע load או store – וזו הנחה לגיטימית.

סיכום מערכות הפעלה | © גיא יער-און

- הפתרון משתמש בשני משתנים משותפים: משתנה שלם turn שיגיד לנו: "התור של מי מהתהליכים להכנס עכשיו לקטע הקוד הקריטי", משתנה שני הוא מערך בוליאני בגודל 2: flag – מערך זה משמש לומר האם תהליך מסוים מעוניין להכנס לקטע הקוד הקריטי שלו. אם $flag[i]=true$ הוא מעוניין להכנס.

```
do {  
    flag[i] = true;  
    turn = j;  
    while (flag[j] && turn == j);  
    critical section  
    flag[i] = false;  
    remainder section  
} while (true);
```

- נתבונן בקטע הקוד עבור תהליך $P[i]$ של האלגוריתם: ראשית הוא מעדכן שהוא מעוניין להכנס לקטע הקוד הקריטי – ובאופן אוטומטי הוא מעביר את התור לצד השני: לתהליך j . בהמשך: כל עוד שזה גם התור של התהליך השני, וגם התהליך השני רוצה להכנס לקטע הקוד הקריטי שלו – אז תישאר במקום. מספיק שאחד מהם לא יתקיים (זה לא התור שלו, או $flag[j]=false$) ואנחנו כן נוכל להכנס. כיצד זה יכול להיות התור של i ? הרי במקביל – j עושה את אותו הדבר, ומעניק לו את התור. כלומר: זה ששיננו את התור עזר לנו. בהמשך – אנחנו משנים את ה- $flag[i]$ ל- $false$ כי אנחנו כבר לא צריכים את קטע הקוד הקריטי.
- שלושת התנאים מתקיימים באלגוריתם של פטרסון:

1. **טענה:** מתקיים באלגוריתם ה- $mutual\ exclusion$ (לא יותר מתהליך אחד יריץ את קטע הקוד הקריטי שלו בו זמנית).

הוכחה: נניח בשלילה שגם P_0 וגם P_1 מריצים את קטע הקוד הקריטי שלהם. זה גורר ששניהם הריצו את $while$ ועברו אותו. אז, P_0 נמצא בקטע הקוד הקריטי שלו וכן $turn=i$ והבדיקה עבור j הייתה שלילית ולכן: או $flag[j]=false$ או $turn=i$. בדומה, P_1 נמצא בקטע הקוד הקריטי שלו ולכן הבדיקה עבור i הייתה שלילית ולכן או $flag[i]=false$ או $turn=j$. אבל אנחנו יודעים כי $flag[i]=flag[j]=true$ שכן הם נמצאים בפנים. לכן בהכרח נקבל כי $turn=i=j$ אבל $i \neq j$ בסתירה.

1. Progress: ישנן 2 אפשרויות:

אפשרות ראשונה – תהליך אחד מבקש להכנס לקטע הקוד הקריטי שלו, והשני לא נמצא בקטע הקוד הקריטי שלו: לכן לפי האלגוריתם התהליך יכנס מיידית לקטע הקוד הקריטי שלו.

אפשרות שנייה – שני התהליכים מבקשים להכנס לקטע הקוד הקריטי שלהם. אך רק אחד יצליח להיות האחרון שיעביר את התור לצד השני – ולכן תהיה בהכרח החלטה מי מבניהם יכנס.

2. Bounded waiting: נבחין כי יש חסם של 1 על מס' הצעדים. לא יתכן ונזכר להכנס לקטע הקוד שלו עד ש- i יסיים אם שניהם ביקשו בו"ז.

פתרונות מבוססי חומרה:

רוב הפתרונות מבוססי החומרה מבוססים על רעיון של locking: הגנה על ה- $critical\ section$ באמצעות מנעול.

כיצד יראה פתרון מבוסס lock? כשננסה להכנס לקטע הקוד הקריטי – ננסה להשתמש ב- $lock$ וברגע שנסיים את קטע הקוד הקריטי: נבצע $release$ ל- $lock$ – ותהליכים אחרים יוכלו להשתמש בו.

ברוב המערכות המודרניות – כבר יש הרבה instructions אטומים: שנעולים באמצעות מנעול. ישנן 2 מרכזיות שנדון בהן ונראה כיצד ניתן לממש $lock$ באמצעותן:

סיכום מערכות הפעלה | © גיא יער-און

```
boolean test_and_set (boolean *target)
{
    boolean rv = *target;
    *target = TRUE;
    return rv;
}

do {
    while (test_and_set(&lock))
        ; /* do nothing */
        /* critical section */
    lock = false;
        /* remainder section */
} while (true);
```

```
do {
    key = TRUE;
    while (key == TRUE)
        Swap (&lock, &key );
    // critical section
    lock = FALSE;
    // remainder section
} while ( TRUE);
```

Test_and_set: פעולה שמבצעים על משתנה בוליאני ומחזירים משתנה בוליאני. מה קורה כאן? מעבירים לפונקציה ערך בוליאני. הפונקציה משנה את ערך המשתנה ל-True ומחזירה את הערך הקודם שהיה למשתנה הבוליאני. מה שמובטח – בעת שהפעלנו את הפעולה: כל מי שיבצע Test_and_set לאחריו יראה ערך של True בוודאות. מדובר בפעולה אטומית (רק אחד יכול לבצע).

בפונקציה זו אנחנו נשתמש על מנת לכתוב פונקציית lock: נראה כי מובטח לנו באמצעות הפונקציה הזו שאנחנו הראשונים שנכנסו וראינו כי ערך המשתנה הוא true, שכן מבצעים test_and_set וזו מתבצעת בצורה אטומית. לכן, אחד התהליכים (הראשון) יריץ אותה ויקבל true ואכן יכנס. בסיום – הוא ישנה את lock ל-false, ולכן הוא משחרר את המנעול עבור תהליכים אחרים: וזה מבטיח שמיד לאחריו תהליך אחר יוכל להשתמש בו.

Swap: קוד אטומי נוסף הוא swap: ללא הפרעה.

גם בקטע קוד זה ניתן להשתמש על מנת לממש lock(): כל תהליך שמנסה להכנס לקטע הקוד הקריטי יוצר לעצמו משתנה מקומי key=True. כעת, כל עוד ערך זה באמת "אמת" הפונקציה Swap מחליפה בצורה אטומית בין הערך של lock לזה של key. ישנם שני תרחישים:

1. אם המנעול היה false (פנוי): אזי key=False והלולאה תעצר, התהליך יכנס לקטע הקוד הקריטי והמנעול יינעל (יהפוך ל-True, כי key=True וביצענו החלפה). כך ששאר התהליכים יישארו תקועים ב-while ולא יכנסו למנעול.

2. אם lock היה כבר אמת – כלומר נעול, ההחלפה תשאיר אותו true והתהליך ימשיך להסתובב בלולאה עד שהמנעול יתפנה.

בסיום, בעת שהתהליך מסיים את קטע הקוד הקריטי הוא משחרר את המנעול (הופך אותו ל-false) מה שמאפשר לתהליך הבא בתור להכנס.

- נבחין כי שני הפתרונות שהצענו לא מקיימים את עקרון bounded waiting: שכן אם יש כמה תהליכים, ויש תהליך שכל הזמן מאבד זמן מעבד בעת הבדיקה של המנעול – ולכן יתכן מצב של הרעבה של אחד התהליכים. דימוי טוב של דודי: ילד שמבצע בצורה אינסופית מתקן בלונה פארק, כי בכל שלב הילד שלפניו הולך לקנות גלידה, ואז חוזר לתור.

לכן – נרצה ללמוד פתרון שתומך גם בדרישה של bounded waiting: נגדיר משתנים משותפים הבאים: מערך בגודל n בשם waiting –

```
do {
    waiting[i] = true;
    key = true;
    while (waiting[i] && key)
        key = test_and_set(&lock);
    waiting[i] = false;
    /* critical section */
    j = (i + 1) % n;
    while ((j != i) && !waiting[j])
        j = (j + 1) % n;
    if (j == i)
        lock = false;
    else
        waiting[j] = false;
    /* remainder section */
} while (true);
```

בוליאני. ששומר לכל תהליך אם הוא רוצה להכנס לקטע הקוד הקריטי שלו. וכן משתנה בוליאני משותף נוסף בשם lock.

הפתרון הנ"ל מונע מצב של הרעבה ומחולק לכמה חלקים:

1. **בעת הכניסה** – תהליך i מסמן שהוא מחכה waiting[i]=true ומנסה לתפוס את המנעול באמצעות test_and_set. הלולאה while תמשיך לרוץ כל עוד הוא מחכה ומישהו אחר מחזיק במנעול – בעת שהוא מצליח, הוא משנה את waiting[i]=false ונכנס לקטע הקוד הקריטי שלו.

2. **בעת יציאה**: לאחר שביצע את קטע הקוד הקריטי שלו, התהליך לא משחרר את המנעול ישירות. התהליך מחפש בצורה ציקלית מעגלית את התהליך הבא בתור שבאמת מחכה – waiting[j]=true. אם נמצא תהליך שכזה – "נעביר לו את המקל"

סיכום מערכות הפעלה | © גיא יער-און

ע"י שינוי הwaiting[j] שלו לfalse. כעת התהליך הבא יפסיק את הwhile שלו ויכנס. אך, אם לא נמצא אף תהליך שמחכה (j=i) שזהו מצב שעשינו סיבוב ציקלי שלם, נשחרר את המנעול.

כעת נשים לב כי כל התנאים אכן מתקיימים, שכן:

- Mutual exclusion: כל תהליך שיוצא מקטע הקוד הקריטי יכול להעביר רק אחד מwaiting[i]=true. וכן רק תהליך אחד יכול לתפוס את המנעול אם הוא משוחרר – כלומר מבין מי שממתין כולם למעט אחד יהיו ללא המנעול ועם waiting=true. לכן רק אחד יכול להשתמש.
- Progress: תהליך שיוצא מקטע הקוד הקריטי יעביר את המנעול לfalse או שיקבע waiting[i]=false לאחד התהליכים הממתנים – ולכן אחד התהליכים הממתנים יוכל להתקדם.
- Bounded waiting: המעבר הציקלי על רשימת התהליכים בעת היציאה מקטע הקוד הקריטי מבטיח שבמקסימום n-1 תהליכים אחרים יריצו את קטע הקוד הקריטי שלהם מהרגע שאנחנו ביקשנו לרוץ עד שרצנו. (במקרה ה"גרוע" ביותר בכל שלב הוא נותן לאחד אחריו בתור ולכן במקסימום תהליך יחכה n-1 סבבים).

Mutex Locks

```
acquire() {
    while (!available)
        ; /* busy wait */
    available = false;;
}
release() {
    available = true;
}
do {
    acquire lock
    critical section
    release lock
    remainder section
} while (true);
```

- Mutex() זה פתרון תוכנה לבעיית הקוד הקריטי. הוא ממומש ע"י מערכת ההפעלה, אך דורש תמיכה של חומרה שכן המימוש דורש פקודות אטומיות – כמו test and set.
- mutex הוא משתנה בוליאני שמשמש כמנעול. אופן השימוש מתבצע כך: תפיסת מנעול באמצעות acquire() וrelease() משחרר את המנעול.

Semaphore

```
wait(S) {
    while (S <= 0)
        ; // busy wait
    S--;
}
signal(S) {
    S++;
}
```

indivisible

- Semaphore הוא משתנה integer שעוזר לסינכרוניזציה. ניתן לבצע עליו שתי פעולות אטומיות –

1. Wait: בעת שערך הסמפור קטן או שווה לאפס – נשארים במקום. כאשר הערך שלו חיובי ממש: ניתן להתקדם ולהוריד את הערך שלו באחד.

2. Signal: מעלה באחד את הערך של הסמפור.

- נעיר, למרות שטענו כי הפעולות אטומיות: רק החלקים של S--, S++ הם אטומיים.

- לסמפור גם נקרא counting semaphore: כיוון שמדובר בערך שלם ולא בוליאני, ניתן

להשתמש בו על מנת להגביל מספרית את מס' התהליכים שיכולים לגשת לבצע דבר מסוים. למשל – אם יש לנו פיזית 5 מדפסות, נוכל להגדיר סמפור שמאותחל לערך 5 שידאג שלא יתכן כי יותר מ-5 תהליכים ינסו להדפיס (שהרי אין מדפסות).

- אם הסמפור בינארי, כלומר מקבל ערכים מ{0,1} נקרא לו mutex – בדיוק אותו דבר.

- Busy waiting: מצב שיש לולאה וכל הזמן אנחנו מבצעים בה בדיקות. מדובר בבזבוז של משאבים! מה יכולה להיות חלופה לזה?

ניתן לבצע לעצמנו Blocking ומישהו מבחוץ יחזיר אותי לזמן מעבד כשנוכל. כלומר נרצה לשנות את הסמפור כפי שהגדרנו

שיוכל לנהל מעין מחסנית של תהליכים – שימתינו עד שהערך של הסמפור יחזור להיות חיובי, והם יכולים להשתמש בו. כך לא

יהיה busy waiting. לשם כך נצטרך 2 פעולות:

```
typedef struct{
```

```
int value;
```

```
struct process *list;
```

```
} semaphore;
```

- 1. Block: לוקחים את התהליך שמחכה לתוך תור waiting queue.

- 2. Wakeup: מעירים את התהליך ומבטלים את הBLOCK.

כעת בהתאמה, נשנה את wait(),signal():

```
wait(semaphore *S) {
    S->value--;
    if (S->value < 0) {
        add this process to S->list;
        block(); ← מעבר למצב waiting
    }
}

signal(semaphore *S) {
    S->value++;
    if (S->value <= 0) {
        remove a process P from S->list;
        wakeup(P); ← חזרה למצב ready
    }
}
```

- signals (שחרור משאב): מגדילים את ערך הסמפור כמו קודם. אם הערך של value הפך לשלילי או אפס, זה גורר כי יש עוד תהליכים שמחכים בתור למשאב שהרגע שוחרר. לכן, מוציאים תהליכים מהתור ומעירים אותו – מבטלים את הBLOCK.
- Wait (לקיחת משאב): מקטינים את הערך. אם הערך שלו הופך לשלילי ממש, זה אומר שלא נותרו משאבים פנויים ולכן הוא נכנס לתור ועובר למצב חסום – Block. אם הערך היה חיובי הוא היה ממשיך בדרכו.

בעיות קלאסיות של סינכרוניזציה

הערה: נפתור את כל הבעיות באמצעות סמפור. **דודי יותר מרמז שבמבחן הוא יבקש לפתור בדרכים אחרות.**

The Bounded-Buffer Problem

בעיית היצרן-צרכן. נתון: buffer בגודל n מקומות ושני תהליכים: אחד כותב לתוך buffer ואחד קורא ממנו.

נשתמש בסמפור מסוג mutex על מנת להגן על buffer מגישה בו זמנית (נאחל את הערך שלו לאחד).

וכן, נשתמש בcounting semaphores על מנת לייצג את מס' המקומות הריקים והמלאים שישנם בבאפר:

```
do {
    ...
    // produce an item in nextp
    ...

    wait(empty);
    wait(mutex);

    // add nextp to buffer

    signal(mutex);
    signal(full);
} while (TRUE);

do {
    wait(full);
    wait(mutex);

    // remove an item from buffer
    // to nextc

    signal(mutex);
    signal(empty);

    // consume the item in nextc
} while (TRUE);
```

1. אחד יהיה empty – ייצג את מס' המקומות הריקים ויאוחלל לנ.
 2. Full – ייצג את מס' המקומות שמולאו ויאוחלל ל0.
- היצרן, משמאל: בתחילה הוא תופס מקום ריק, ונועל את הבאפר wait(mutex) על מנת לעבוד לבד ושהצרכן לא יגע בו. לאחר מכן, מתבצעת הכתיבה לבאפר. בסיום: משחרר את הנעילה של הבאפר, וכן מגדיל את הערך של full (נוסף מקום אחד).

הצרכן, מימין: ראשית הוא בודק אם יש פריט - אם הבאפר ריק (full=0) הוא נחסם ומחכה. שהרי הוא צרכן ולא יכול לקחת פריטים אם אין כאלו. בהמשך, הוא נכנס ונועל את הבאפר על מנת לקרוא ממנו לבד ללא הצרכן. הוא קורא מהבאפר. לאחר מכן - משחרר את נעילת הבאפר, ומעדכן שעכשיו יש מקום אחד פנוי נוסף (מעדכן את empty): מקטין אותו באחד. נבהיר - הוא קודם כל בודק אם יש מקום / אין מקום ורק אז נועלים את הבאפר.

The Readers-Writers Problem

סיכום מערכות הפעלה | © גיא יער-און

נניח שיש לנו דטה בייס ותהליכים שיש להם גישה אליו. חלק מהתהליכים רק קוראים מהדטה בייס – יקראו readers, וכן אלו שכותבים וגם קוראים מהדטה בייס יקראו writers. אין כל בעיה במצב שבו יותר מ-1 reader אחד ניגש ל-DB.

מה הבעיה? אם writer ותהליך נוסף (כלשהו, קורא או כותב) ניגשים במקביל ל-DB עלול להיווצר כאוס. נבחין כי עלינו שלא לגרום לעיכוב תהליכים שלא לצורך.

נחזיר במשתנים משותפים: סמפור בשם mutex: משותף ל-readers וכן נחזיר מנעול wrt מסוג סמפור שיהיה משותף לכולם – גם ל-writers and readers. שניהם יאוחללו ל-1, וכן, נחזיר משתנה שלם בשם readcount שמשותף לכל ה-readers ויאוחלל ל-0. (יספור כמה readers קוראים כעת).

הקורא, מימין: הקורא ראשית מקדם את readcount – אם זה הקורא הראשון הוא נועל את בסיס הנתונים בפני הכותבים שלא יוכלו לכתוב. שאר הקוראים שיבואו אחריו לא יבצעו נעילה, שכן הוא כבר בוצע וערך המשתנה גדול ממש מאחד. הקוראים מבצעים קריאה, ובסיום: הקורא מעדכן את readcount שירד באחד ובדומה. אם זה הקורא האחרון – הוא משחרר את בסיס הנתונים מה שיאפשר לכותבים לכתוב בו.

הכותב, משמאל: מנסה להשתמש במשאב הכתיבה – אם כבר יש קוראים או כותבים בפנים הוא יחסם. ביציאה – משחרר את הנעילה ומאפשר לאחרים להכנס.

The Dining-Philosopher Problem

```
do {
    wait(chopstick[i]);
    wait(chopstick[(i+1) % 5]);

    ...
    // eat
    ...

    signal(chopstick[i]);
    signal(chopstick[(i+1)%5]);

    ...
    // think
    ...
}while (TRUE);
```

הבעיה מוגדרת כך: חמישה פילוסופים מעבירים את חייהם בישיבה סביב שולחן כשהם אחד בשני מצבים: הוגים או אוכלים. לא ישנים! השולחן מסודר כך שבמרכזו קערת אוכל וסביבו 5 מקלות אכילה (בשביל לאכול צריך 2 מקלות). כאשר פילוסוף נהיה רעב, הוא מנסה להרים מהשולחן את שני המקלות הקרובים ביותר אליו. לא ניתן להרים שני מקלות בו זמנית – רק אחד בכל פעם. וכן: לא ניתן לקחת מקל אם הוא נמצא בידי פילוסוף סמוך. בעת שפילוסוף שבע, הוא מניח את המקלות על השולחן וחוזר להגות. כיצד נפתור זאת?

ניצור מערך chopstick מסוג סמפור בגודל 5 – שמשותף לכל התהליכים (פילוסופים). בהתחלה הוא יאוחלל ל-1 כל איבריו. בעת שצ'ופסטיק נתפס – הוא תופס (wait) את הצ'ופסטיק שמצא ואת האחד אחריו (במעגל). הוא אוכל – ובעת שמסיים: הוא מבצע signal על שני הצ'ופסטיקים.